Embedded Module Architecture 実装ガイド

リポジトリの各モジュールで学ぶ設計パターンの実践

バージョン 1.3

作成日 2026 年 4 月 12 日

著者 Shiozawa

本ドキュメントは、Embedded Module Architecture リポジトリに
実装されている各モジュールを題材に、
設計パターンの適用方法を具体的なコードで解説するガイドです。

設計仕様書（設計ルール）・教科書（パターンの学習）と併せてお読みください。

改訂履歴

版	日付	著者	変更内容
1.0	2026-03-15	Shiozawa	初版作成。全 13 モジュールの実装解説
1.1	2026-03-17	Shiozawa	関連ドキュメントの相互参照を追加。日付・Rev 番号を全ドキュメントと同期
1.2	2026-03-18	Shiozawa	ESP32 固有の記述を Arduino/組み込み汎用に変更。ADC コメント・IRAM_ATTR 注記等を汎用化
1.3	2026-04-12	Shiozawa	各モジュール解説に API 仕様（Config/Data 構造体・メソッド・使用例）を追加。アーキテクチャ仕様書から移動

目次

改訂履歴 1

1 この資料について 4
- 1.1 位置づけ . . . 4
- 1.2 読み方 . . . 4
- 1.3 モジュール一覧と学習レベル . . . 4

2 【基本】LedModule — 最小構成の基本パターン 5
- 2.1 ヘッダー（LedModule.h） . . . 5
- 2.2 実装（LedModule.cpp） . . . 6
- 2.3 ProjectConfig.h での定義 . . . 6
- 2.4 API 仕様 . . . 7

3 【基本】ButtonModule — デバウンスパターン 8
- 3.1 Config/Data 構造体 . . . 8
- 3.2 デバウンスの実装 . . . 9
- 3.3 ロジックフェーズでの使用例 . . . 10
- 3.4 API 仕様 . . . 10

4 【基本】BatteryModule — ADC + 移動平均フィルタ 12
- 4.1 Config/Data 構造体 . . . 12
- 4.2 リングバッファと移動平均 . . . 13
- 4.3 API 仕様 . . . 14

5 【基本】ServoModule — PWM 出力パターン 16
- 5.1 Config/Data 構造体 . . . 16
- 5.2 updateOutput() の実装 . . . 17
- 5.3 API 仕様 . . . 17

6 【基本】BuzzerModule — deinit() パターン 20
- 6.1 Data 構造体のリクエストパターン . . . 20
- 6.2 updateOutput() の実装 . . . 20
- 6.3 deinit() の実装 . . . 21
- 6.4 API 仕様 . . . 21

7 【中級】Mpu6500Module — I2C バスパターン 24
- 7.1 バスポインタの受け取り . . . 24
- 7.2 main.cpp でのバス管理 . . . 24
- 7.3 API 仕様 . . . 25

8 【中級】DisplayBoardModule — 入出力統合 + ライブラリ内部管理 28
- 8.1 なぜ 1 モジュールに統合するのか . . . 28
- 8.2 ライブラリの内部管理パターン . . . 28
- 8.3 Data 構造体の入出力統合 . . . 29

8.4 ロジックフェーズでの表示内容設定 29
8.5 API 仕様 30

9 【中級】CameraModule — PSRAM バッファ + フレーム管理 33
9.1 Config の大量ピン管理 33
9.2 フレームバッファの管理 33
9.3 API 仕様 34

10 【応用】DriveMotorModule — PWM 2 ピン制御 + 統合部品 37
10.1 2 つの使い方 38
10.2 逆転保護 38
10.3 API 仕様 39

11 【応用】ChassisModule — 機能統合パターン 41
11.1 内部モジュールの保持 41
11.2 親モジュールからの制御委譲 42
11.3 API 仕様 44

12 【応用】BleModule — スレッドセーフティパターン 47
12.1 内部バッファと volatile フラグ 47
12.2 コールバック→内部バッファ→ SystemData 48
12.3 BLE コールバックの登録 49
12.4 送信の流れ（ロジックフェーズ→ updateOutput()） 49
12.5 API 仕様 50

13 【応用】WifiModule — ステートマシンパターン 53
13.1 状態定義 53
13.2 ステートマシンの実装 54
13.3 ロジックフェーズでの使用例 55
13.4 API 仕様 56

14 【応用】EncoderModule — 割り込み連携パターン 59
14.1 割り込みハンドラの設計 59
14.2 ISR → volatile → updateInput() の流れ 60
14.3 deinit() で割り込みを解除 61
14.4 API 仕様 62

1 この資料について

1.1 位置づけ

本リポジトリには 2 つのプロジェクト（`sample/`: サンプル実装、`verified/`: 動作確認済み）と 3 つのドキュメントがある。

ドキュメント	ディレクトリ	内容
設計仕様書	`doc/`	アーキテクチャのルール・API 仕様（「何を守るか」）
教科書	`doc-textbook/`	パターンの段階的な学習（「なぜそうするか」）
実装ガイド（本書）	`doc-guide/`	実コードの解説（「実際にどう書いたか」）

！ サンプルコードについて

`sample/` に実装されているモジュール群のコードは、あくまで**アーキテクチャの設計パターンを具体的に示すためのサンプル実装**である。実機での動作検証は行っていないため、実プロジェクトへの適用時は十分なテストを実施すること。動作確認済みのコードは `verified/` を参照すること。

1.2 読み方

本書では、リポジトリに実装されている各モジュールを取り上げ、以下の構成で解説する。

1. **概要** — モジュールの役割と、そのモジュールで学べるパターン
2. **コード解説** — 実際のコードを引用し、設計ルールとの対応を示す
3. **ポイント** — 特に注目すべき設計判断やテクニック

モジュールは簡単なものから順に並べている。前半のモジュールで基本パターンを押さえたうえで、後半の応用パターンに進む構成になっている。

1.3 モジュール一覧と学習レベル

各モジュールには難易度レベルを付けている。まず【基本】を全て読んでパターンを身につけてから、必要に応じて【中級】【応用】に進むことを推奨する。

モジュール	種別	レベル	主な学習パターン
LedModule	出力	【基本】	最小構成の基本パターン
ButtonModule	入力	【基本】	デバウンス（ModuleTimer 活用）
BatteryModule	入力	【基本】	ADC 読み取り ＋ 移動平均フィルタ
ServoModule	出力	【基本】	PWM 制御（LEDC）
BuzzerModule	出力	【基本】	deinit() によるリソース解放
Mpu6500Module	入力	【中級】	I2C バスパターン ＋ レジスタ直接アクセス
DisplayBoardModule	入出力	【中級】	入出力統合 ＋ ライブラリ内部管理
CameraModule	入力	【中級】	PSRAM バッファ ＋ フレーム管理
DriveMotorModule	出力	【応用】	PWM 2 ピン制御 ＋ 統合モジュール部品
ChassisModule	出力	【応用】	機能統合系（内部にモジュールを持つ）
BleModule	入出力	【応用】	スレッドセーフティ（volatile フラグ方式）

モジュール	種別	レベル	主な学習パターン
WifiModule	入出力	【応用】	ステートマシン内蔵
EncoderModule	入力	【応用】	ハードウェア割り込み連携

▷ レベルの目安

- **【基本】** — 教科書の内容だけで理解できる。最初に全て読むべき
- **【中級】** — I2C/SPI 等の通信バスパターンが加わる。必要に応じて読む
- **【応用】** — 統合パターン・スレッドセーフティ・割り込みなど高度な内容。チャレンジ向け

2 【基本】LedModule — 最小構成の基本パターン

LedModule で学べるパターン

- Config 構造体によるピンアサインの外部注入
- Data 構造体のデフォルト値
- .h/.cpp 分離（循環依存回避）
- 出力モジュールの基本形（SystemData の値に従って動作するだけ）

2.1 ヘッダー（LedModule.h）

```
#pragma once
#include <Arduino.h>
#include "IModule.h"

struct LedConfig {
    uint8_t ledPin;  // LED出力ピン
};

struct LedData {
    bool state = false;        // LED状態（デフォルト：消灯）
    uint8_t brightness = 0;    // 輝度（将来拡張用）
};

class LedModule : public IModule {
private:
    LedConfig _config;
public:
    LedModule(const LedConfig& config);
    bool init() override;
    void updateOutput(SystemData& data) override;
};
```

Listing 1: LedModule.h

✓ Config 構造体のポイント

LedConfig にはピン番号だけを持つ。ピン番号を LedModule 内にハードコードせず、Config 構造体で外部から注入する。これにより、同じ LedModule クラスを別プロジェクトでピンだけ変えて再利用できる。

✓ Data 構造体のデフォルト値

LedData の state = false は「updateOutput() が 1 度も呼ばれていない状態でも安全」を保証する。ロジックフェーズが state を true に変更するまで、LED は消灯したままとなる。

2.2 実装（LedModule.cpp）

```cpp
#include "LedModule.h"
#include "SystemData.h"  // ヘッダーでは前方宣言のみ、cppで完全な定義を取得

LedModule::LedModule(const LedConfig& config) : _config(config) {}

bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);
    Serial.println("[Led] init OK");
    return true;
}

void LedModule::updateOutput(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state ? HIGH : LOW);
}
```

Listing 2: LedModule.cpp

✓ 出力モジュールの責務

updateOutput() は**自身の Data 構造体の値に従って動作するだけ**。「いつ点灯するか」「何の条件で消すか」などのロジックは一切書かない。それはロジックフェーズ（applyPattern() 等）の仕事である。

✓ .h/.cpp 分離のルール

ヘッダーでは SystemData.h をインクルードせず、IModule.h 内の前方宣言だけを利用する。SystemData.h のインクルードは.cpp 側で行う。これにより LedModule.h → SystemData → LedModule.h の循環依存を防ぐ。

2.3 ProjectConfig.h での定義

```cpp
const LedConfig LED_CONFIG = {
    .ledPin = 2,  // GPIO2: オンボードLED
};
```

Listing 3: ProjectConfig.h での Config 定義

回路設計者はこのファイルだけ見ればピンアサインが分かる。LED_CONFIG という大文字スネークケースの命名規則も守っている。

2.4 API 仕様

LedModule — LED 制御モジュール

定義ファイル: `lib/LedModule/LedModule.h / .cpp`
分類: 出力モジュール（`updateOutput()` のみオーバーライド）
対象デバイス: 汎用 LED
GPIO 出力により LED の ON/OFF を制御するシンプルなモジュール。本アーキテクチャの最小実装例として位置づけられる。

2.4.1 機能概要

- GPIO 出力による LED ON/OFF 制御
- `state` フラグによるシンプルな状態管理
- `brightness` メンバ（将来の PWM 調光用に予約）

2.4.2 Config 構造体

◆ `LedConfig`

LED 接続ピンを定義する。

メンバ	型	制約・既定値	説明
`ledPin`	`uint8_t`	—	LED 出力 GPIO ピン番号

2.4.3 Data 構造体

◆ `LedData`

LED の出力状態を保持する。

メンバ	型	初期値	説明
`state`	`bool`	`false`	`true`: LED 点灯、`false`: LED 消灯
`brightness`	`uint8_t`	0	輝度値（0–255）。現在の実装では GPIO の HIGH/LOW のみで未使用

2.4.4 メソッド

◆ `LedModule(const LedConfig& config)`

項目	内容
引数	`config` — `LedConfig` への const 参照
動作	Config を `_config` にコピー保持する

◆ `bool init()`

項目	内容
戻り値	`bool` — 常に `true`
動作	`ledPin` を `OUTPUT` モードに設定し、LOW に初期化する

◆ `void updateOutput(SystemData& data)`

項目	内容
読み取り元	`data.led`
タイミング制御	毎ループ実行
動作	`data.led.state` に応じて `digitalWrite()` で HIGH/LOW を出力する

2.4.5 使用例

```
// ProjectConfig.h
const LedConfig LED_CONFIG = {
    .ledPin = 2,  // GPIO2: オンボードLED
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // ボタン押下でLEDをトグル
    if (data.button.justPressed) {
        data.led.state = !data.led.state;
    }
}
```

Listing 4: LedModule の使用例

3 【基本】ButtonModule — デバウンスパターン

ButtonModule で学べるパターン

- ModuleTimer を使ったデバウンス処理
- エッジ検出（justPressed / justReleased）
- 入力モジュールの基本形（ハードウェアから読んで SystemData に書くだけ）

3.1 Config/Data 構造体

```
struct ButtonConfig {
    uint8_t       pin;          // ボタンピン
    bool          activeLow;    // true: LOW=押下（プルアップ）
    unsigned long debounceMs;   // デバウンス安定時間 [ms]
};

```

```
struct ButtonData {
    bool pressed      = false;  // デバウンス後の押下状態
    bool justPressed  = false;  // 今ループで押下された（立ち上がりエッジ）
    bool justReleased = false;  // 今ループで離された（立ち下がりエッジ）
};
```

Listing 5: ButtonModule.h — Config/Data

✓ Config に動作パラメータを含める

debounceMs はピンアサインではないが、モジュールの動作特性を決める設定なので Config に含める。ハードウェア環境によって最適なデバウンス時間は異なるため、外部から注入可能にしておく。

3.2 デバウンスの実装

```
void ButtonModule::updateInput(SystemData& data) {
    // 生の入力値を読み取り
    bool rawPressed = digitalRead(_config.pin);
    if (_config.activeLow) {
        rawPressed = !rawPressed;
    }

    // 入力値が変化したらデバウンスタイマーをリセット
    if (rawPressed != _lastRaw) {
        _debounceTimer.setTime();
        _lastRaw = rawPressed;
    }

    // デバウンス安定時間を超えたら状態を確定
    if (_debounceTimer.getNowTime() >= _config.debounceMs) {
        _stableState = _lastRaw;
    }

    // エッジ検出
    data.button.pressed      = _stableState;
    data.button.justPressed  = (_stableState && !_prevStable);
    data.button.justReleased = (!_stableState && _prevStable);
    _prevStable = _stableState;
}
```

Listing 6: ButtonModule.cpp — updateInput()

✓ ModuleTimer によるデバウンス

仕様書のデバウンスパターンそのまま。入力が変化するたびに setTime() でタイマーをリセットし、debounceMs の間安定したら状態を確定する。delay() は一切使わず、毎ループ非同期で判定している。

✓ エッジ検出を Data 構造体に含める

justPressed と justReleased を Data 構造体に含めることで、ロジックフェーズで「ボタンが今押された瞬間」を簡単に検出できる。if (data.button.justPressed) と書くだけでよく、ロジック側で前回値を保持する必要がない。

3.3 ロジックフェーズでの使用例

```
void applyPattern(SystemData& data) {
    // ボタンが押された瞬間にLEDをトグル
    if (data.button.justPressed) {
        data.led.state = !data.led.state;
    }

    // ボタン押下中にブザーを鳴らす
    if (data.button.pressed) {
        data.buzzer.frequency = 1000;
        data.buzzer.requestPlay = true;
    }
}
```

Listing 7: ロジックフェーズでの ButtonData 参照

3.4 API 仕様

ButtonModule — ボタン入力モジュール

定義ファイル: `lib/ButtonModule/ButtonModule.h / .cpp`
分類: 入力モジュール（`updateInput()` のみオーバーライド）
対象デバイス: 汎用タクトスイッチ・プッシュボタン
GPIO 入力をデバウンス処理し、押下状態とエッジ（立ち上がり・立ち下がり）を検出するモジュール。プルアップ接続（activeLow）とプルダウン接続の両方に対応する。

3.4.1 機能概要

- デバウンスフィルタ（`ModuleTimer` ベース、設定可能な安定時間）
- プルアップ（LOW=押下）/ プルダウン（HIGH=押下）の切り替え対応
- エッジ検出フラグ（`justPressed` / `justReleased`）
- 安定状態の保持（`pressed`）

3.4.2 Config 構造体

◆ `ButtonConfig`

ボタンのハードウェア接続とデバウンスパラメータを定義する。

メンバ	型	制約・既定値	説明
`pin`	`uint8_t`	—	ボタン入力 GPIO ピン番号
`activeLow`	`bool`	—	`true`: LOW=押下（プルアップ接続）、`false`: HIGH=押下（プルダウン接続）
`debounceMs`	`unsigned long`	例: 30	デバウンス安定時間 [ms]。入力変化後、この時間が経過するまで状態を確定しない

3.4.3 Data 構造体

◆ ButtonData

デバウンス処理後のボタン状態を保持する。justPressed/justReleased は 1 ループ限りのエッジフラグであり、次の updateInput() 呼び出しでクリアされる。

メンバ	型	初期値	説明
pressed	bool	false	デバウンス後の押下状態。押されている間 true
justPressed	bool	false	今ループで押下された瞬間のみ true（立ち上がりエッジ）
justReleased	bool	false	今ループで離された瞬間のみ true（立ち下がりエッジ）

3.4.4 メソッド

◆ ButtonModule(const ButtonConfig& config)

項目	内容
引数	config — ButtonConfig への const 参照
動作	Config を_config にコピー保持する

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	activeLow に応じて INPUT_PULLUP または INPUT でピンモードを設定。デバウンスタイマーを初期化

◆ void updateInput(SystemData& data)

項目	内容
書き込み先	data.button
タイミング制御	毎ループ実行（タイマーはデバウンス判定に使用）
動作	digitalRead() で生入力を取得し、前回値と比較。変化があればデバウンスタイマーをリセット。タイマー経過後に安定状態を確定し、pressed・justPressed・justReleased を更新

▷ エッジフラグのライフサイクル

justPressed と justReleased は updateInput() の先頭で毎回 false にリセットされる。状態変化があったループでのみ true になるため、ロジックフェーズで 1 回だけ検出できる。複数箇所で参照しても安全である（同一ループ内では値が変わらない）。

3.4.5 使用例

```
// ProjectConfig.h
const ButtonConfig BUTTON_CONFIG = {
    .pin        = 0,      // GPIO0: BOOTボタン
    .activeLow  = true,   // LOW=押下（内蔵プルアップ使用）
    .debounceMs = 30,     // デバウンス30ms
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // ボタンが押された瞬間にLEDをトグル
    if (data.button.justPressed) {
        data.led.state = !data.led.state;
    }
}
```

Listing 8: ButtonModule の使用例

4 【基本】BatteryModule — ADC + 移動平均フィルタ

BatteryModule で学べるパターン

- ModuleTimer による周期サンプリング
- リングバッファを使った移動平均フィルタ
- ADC 値の電圧変換（分圧回路対応）
- isValid フラグによる「データが十分溜まったか」の表現

4.1 Config/Data 構造体

```
static constexpr uint8_t BATTERY_FILTER_SIZE = 16;

struct BatteryConfig {
    uint8_t       adcPin;
    float         voltageDivider;      // 分圧比（例: 2.0 = 1/2分圧）
    float         adcRefVoltage;       // ADC基準電圧 [V]
    uint16_t      adcResolution;       // ADC分解能（例: ESP32は4095、AVRは1023）
    unsigned long sampleIntervalMs;
    float         lowVoltageThreshold; // 低電圧警告閾値 [V]
};

struct BatteryData {
    float voltage    = 0.0f;   // フィルタ済み電圧 [V]
    float rawVoltage = 0.0f;   // 最新の生電圧 [V]
    bool  isValid    = false;  // サンプルが十分溜まったらtrue
    bool  isLow      = false;  // 低電圧警告フラグ
};
```

Listing 9: BatteryModule.h — Config/Data

✓ Config にハードウェア特性を含める

voltageDivider（分圧比）や adcRefVoltage（基準電圧）はハードウェア構成に依存する値。これらを Config に含めることで、異なる回路構成でも同じモジュールコードを再利用できる。

4.2 リングバッファと移動平均

```
void BatteryModule::updateInput(SystemData& data) {
    if (_sampleTimer.getNowTime() < _config.sampleIntervalMs) {
        return;  // 周期未到達なら何もしない（ノンブロッキング）
    }
    _sampleTimer.setTime();

    // ADC読み取り -> 電圧変換
    uint16_t adcRaw = analogRead(_config.adcPin);
    float voltage = _adcToVoltage(adcRaw);

    // リングバッファに追加
    _samples[_sampleIndex] = voltage;
    _sampleIndex = (_sampleIndex + 1) % BATTERY_FILTER_SIZE;
    if (_sampleCount < BATTERY_FILTER_SIZE) {
        _sampleCount++;
    }

    // SystemDataに書き込み
    data.battery.rawVoltage = voltage;
    data.battery.voltage    = _calcAverage();
    data.battery.isValid    = (_sampleCount >= BATTERY_FILTER_SIZE);
    data.battery.isLow      = (data.battery.voltage < _config.lowVoltageThreshold)
        ;
}
```

Listing 10: BatteryModule.cpp — updateInput()

✓ isValid の使い方

isValid は「移動平均のサンプルが BATTERY_FILTER_SIZE に達したか」を示す。起動直後はサンプルが少なく平均値が不正確なため、ロジックフェーズは isValid == true になるまで電圧値を信用しないようにできる。仕様書の「デフォルト値は正常値と区別できる無効値」ルールと同じ考え方。

✓ 周期サンプリングの定石

updateInput() の冒頭で getNowTime() < interval なら即 return するパターンは、周期実行の定石。ガード節として最初に書くことで、残りのコードのインデントが深くならない。

4.3 API 仕様

BatteryModule — バッテリー電圧監視モジュール

定義ファイル: `lib/BatteryModule/BatteryModule.h / .cpp`
分類: 入力モジュール（`updateInput()` のみオーバーライド）
対象デバイス: ADC 入力（分圧回路経由）
ADC でバッテリー電圧を読み取り、移動平均フィルタで平滑化するモジュール。分圧比の補正、低電圧警告判定を行う。フィルタサイズは 16 サンプル固定。

4.3.1 機能概要

- 周期的 ADC サンプリング（`ModuleTimer` ベース）
- 16 サンプル移動平均フィルタ（リングバッファ実装）
- 分圧回路の補正（`voltageDivider` で実電圧に換算）
- 低電圧警告フラグ（`isLow`）
- サンプル蓄積状態の管理（`isValid`）

◆ 定数

名前	値	説明
`BATTERY_FILTER_SIZE`	16	移動平均フィルタのサンプル数（リングバッファサイズ）

4.3.2 Config 構造体

◆ `BatteryConfig`

ADC 入力と電圧計算のパラメータを定義する。

メンバ	型	制約・既定値	説明
`adcPin`	`uint8_t`	—	ADC 入力 GPIO ピン番号
`voltageDivider`	`float`	例: 2.0	分圧比。実電圧 = ADC 電圧 × この値
`adcRefVoltage`	`float`	ESP32: 3.3	ADC 基準電圧 [V]
`adcResolution`	`uint16_t`	ESP32: 4095	ADC 分解能（12bit = 4095）
`sampleIntervalMs`	`unsigned long`	例: 500	サンプリング間隔 [ms]
`lowVoltageThreshold`	`float`	例: 3.3	低電圧警告閾値 [V]。フィルタ済み電圧がこの値以下で `isLow` が `true`

4.3.3 Data 構造体

◆ BatteryData

フィルタ処理後のバッテリー電圧と状態フラグを保持する。

メンバ	型	初期値	説明
voltage	float	0.0f	移動平均フィルタ済み電圧 [V]
rawVoltage	float	0.0f	最新の生電圧 [V]（フィルタ前）
isValid	bool	false	サンプル数が BATTERY_FILTER_SIZE に達したら true
isLow	bool	false	低電圧警告フラグ

4.3.4 メソッド

◆ BatteryModule(const BatteryConfig& config)

項目	内容
引数	config — BatteryConfig への const 参照
動作	Config を _config にコピー保持。リングバッファをゼロ初期化

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	ADC ピンのモード設定、サンプリングタイマーの初期化

◆ void updateInput(SystemData& data)

項目	内容
書き込み先	data.battery
タイミング制御	ModuleTimer で _config.sampleIntervalMs ごとに実行
動作	ADC 生値を読み取り、電圧に変換してリングバッファに追加。バッファ全体の平均を計算し voltage に書き込む。lowVoltageThreshold と比較して isLow を更新

▷ 電圧変換式

ADC 生値から実電圧への変換は以下の式で行う:

$$V_{actual} = \frac{adcValue}{adcResolution} \times adcRefVoltage \times voltageDivider$$

例: ADC 値 2048、分解能 4095、基準電圧 3.3V、分圧比 2.0 の場合: $V = \frac{2048}{4095} \times 3.3 \times 2.0 \approx 3.30\text{V}$

4.3.5 使用例

```
// ProjectConfig.h
const BatteryConfig BATTERY_CONFIG = {
    .adcPin             = 3,      // GPIO3: ADC1_CH2
    .voltageDivider     = 2.0f,   // 1/2分圧回路
    .adcRefVoltage      = 3.3f,   // ESP32 ADC基準電圧
    .adcResolution      = 4095,   // 12bit ADC
    .sampleIntervalMs   = 500,    // 500msごとにサンプリング
    .lowVoltageThreshold = 3.3f,  // 3.3V以下で低電圧警告
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    if (data.battery.isValid && data.battery.isLow) {
        // 低電圧時の処理（LED点滅やBLE通知など）
        data.led.state = true;
    }
}
```

Listing 11: BatteryModule の使用例

5 【基本】ServoModule — PWM 出力パターン

ServoModule で学べるパターン

- LEDC（PWM）の初期化と制御
- 「変化がある場合のみ更新」による無駄な出力の抑制
- currentAngle による出力状態のフィードバック

5.1 Config/Data 構造体

```
struct ServoConfig {
    uint8_t  pin;            // サーボ信号ピン
    uint8_t  pwmChannel;     // LEDCチャネル番号（0-15）
    uint16_t minPulseUs;     // 最小パルス幅 [us]（例: 500）
    uint16_t maxPulseUs;     // 最大パルス幅 [us]（例: 2500）
    uint8_t  defaultAngle;   // 初期角度 [度]（0-180）
};

struct ServoData {
    uint8_t targetAngle  = 90;  // 目標角度（ロジックフェーズが設定）
    uint8_t currentAngle = 90;  // 現在の角度（モジュールが更新）
};
```

Listing 12: ServoModule.h — Config/Data

✓ target と current の分離

targetAngle はロジックフェーズが書き込む「指示値」、currentAngle はモジュールが更新する「実際の出力値」。この分離により、ロジックフェーズが「今サーボがどの角度にいるか」を確認できる。

5.2 updateOutput() の実装

```
void ServoModule::updateOutput(SystemData& data) {
    uint8_t target = constrain(data.servo.targetAngle, 0, 180);

    // 変化がある場合のみPWM更新
    if (target != data.servo.currentAngle) {
        ledcWrite(_config.pwmChannel, _angleToDuty(target));
        data.servo.currentAngle = target;
    }
}
```

Listing 13: ServoModule.cpp — updateOutput()

✓ 変化検出パターン

毎ループ PWM を書き込むのではなく、target != current の場合のみ更新する。同じ値を毎回書き込んでも動作に問題はないが、不要なハードウェアアクセスを避けるのは良い習慣。特に I2C/SPI 経由のデバイスでは通信コストの削減になる。

✓ constrain による安全性

constrain() でサーボの物理的な可動範囲に制限している。ロジックフェーズが誤って範囲外の値を設定しても、モジュール側で安全に丸める。入力バリデーションはモジュールの責務。

5.3 API 仕様

ServoModule — サーボモーター制御モジュール

定義ファイル: lib/ServoModule/ServoModule.h / .cpp
分類: 出力モジュール（updateOutput() のみオーバーライド）
対象デバイス: PWM サーボモーター（SG90 等）
LEDC による PWM パルス幅制御でサーボモーターの角度を設定するモジュール。角度（0–180°）からパルス幅（μs）への変換を内部で行う。

5.3.1 機能概要

- 角度指定（0–180°）によるサーボ制御
- パルス幅範囲のカスタマイズ（minPulseUs / maxPulseUs）
- 角度変化時のみ PWM 更新（不要な書き込みを抑制）
- 初期角度の設定（defaultAngle）

5.3.2 Config 構造体

◆ ServoConfig

サーボの PWM パラメータを定義する。

メンバ	型	制約・既定値	説明
pin	uint8_t	—	サーボ信号出力 GPIO ピン番号
pwmChannel	uint8_t	0–15	LEDC チャネル番号。他モジュールと重複しないこと
minPulseUs	uint16_t	例: 500	0° 時のパルス幅 [μs]
maxPulseUs	uint16_t	例: 2500	180° 時のパルス幅 [μs]
defaultAngle	uint8_t	例: 90	初期角度 [度]（0–180）

5.3.3 Data 構造体

◆ ServoData

サーボの目標角度と現在角度を保持する。

メンバ	型	初期値	説明
targetAngle	uint8_t	90	目標角度 [度]（0–180）。ロジックフェーズで設定
currentAngle	uint8_t	90	現在の PWM 出力に対応する角度 [度]（0–180）

5.3.4 メソッド

◆ ServoModule(const ServoConfig& config)

項目	内容
引数	config — ServoConfig への const 参照
動作	Config を_config にコピー保持する

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	LEDC チャネルを 50Hz（サーボ標準）で初期設定し、defaultAngle に対応するパルス幅を出力する

◆ `void updateOutput(SystemData& data)`

項目	内容
読み取り元	`data.servo`
タイミング制御	毎ループ実行（ただし角度変化なしの場合は PWM 更新をスキップ）
動作	`targetAngle` と `currentAngle` を比較し、変化があれば角度をパルス幅に変換して PWM デューティを更新。`currentAngle` を更新

▷ 角度→パルス幅変換

角度からパルス幅への変換は以下の線形補間で行う:

$$pulseWidth = minPulseUs + \frac{angle}{180} \times (maxPulseUs - minPulseUs)$$

SG90 の場合（500–2500μs）：90° → 1500μs（中央）

5.3.5 使用例

```
// ProjectConfig.h
const ServoConfig SERVO_CONFIG = {
    .pin          = 1,     // GPIO1: サーボ信号
    .pwmChannel   = 0,     // LEDCチャネル0
    .minPulseUs   = 500,   // SG90: 500us
    .maxPulseUs   = 2500,  // SG90: 2500us
    .defaultAngle = 90,    // 初期角度 90度（中央）
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // IMUの傾きに応じてサーボ角度を設定
    if (data.mpu.isValid) {
        float tilt = atan2(data.mpu.accelX, data.mpu.accelZ);
        data.servo.targetAngle = constrain(
            map(tilt * 57.3, -45, 45, 0, 180), 0, 180
        );
    }
}
```

Listing 14: ServoModule の使用例

6 【基本】BuzzerModule — deinit() パターン

BuzzerModule で学べるパターン

- deinit() による PWM リソースの明示的解放
- リクエスト方式による出力制御（requestPlay フラグ）
- ModuleTimer による時間指定再生の自動停止

6.1 Data 構造体のリクエストパターン

```
struct BuzzerData {
    uint16_t frequency = 0;        // 再生周波数 [Hz]（0=消音）
    unsigned long durationMs = 0; // 再生時間 [ms]（0=無制限）
    bool     requestPlay = false; // ロジックフェーズからの再生リクエスト
};
```

Listing 15: BuzzerModule.h — Data

✓ リクエストフラグパターン

requestPlay は「ロジックフェーズからの指示を 1 回だけ処理する」ためのフラグ。updateOutput() が処理後に false に戻すため、次ループでは再度処理しない。LedModule のように毎ループ同じ状態を出力するのではなく、「イベント的に 1 回だけ何かをする」場合にこのパターンを使う。

6.2 updateOutput() の実装

```
void BuzzerModule::updateOutput(SystemData& data) {
    // 再生リクエスト処理
    if (data.buzzer.requestPlay) {
        data.buzzer.requestPlay = false;  // 処理済みフラグクリア

        if (data.buzzer.frequency > 0) {
            _toneOn(data.buzzer.frequency);
            if (data.buzzer.durationMs > 0) {
                _durationTimer.setTime();
                _timed = true;
            } else {
                _timed = false;  // 無制限再生
            }
        } else {
            _toneOff();
        }
    }

    // 時間指定ありの場合、経過したら自動消音
    if (_playing && _timed) {
        if (_durationTimer.getNowTime() >= data.buzzer.durationMs) {
            _toneOff();
        }
    }
```

```
}
```

Listing 16: BuzzerModule.cpp — updateOutput()

6.3 deinit() の実装

```
void BuzzerModule::deinit() {
    _toneOff();
    ledcDetachPin(_config.pin);  // PWMチャネルとピンの紐付けを解除
    Serial.println("[Buzzer] deinit OK");
}
```

Listing 17: BuzzerModule.cpp — deinit()

✓ deinit() の実装が必要なケース

`IModule::deinit()` はデフォルトで空実装。BuzzerModule のように**ハードウェアリソース（PWM チャネル）を占有する**モジュールでは、スリープ前やモジュール停止時にリソースを明示的に解放する必要がある。
`ledcDetachPin()` を呼ばないと、PWM チャネルが占有されたままスリープに入り、復帰後に同じチャネルを使えなくなる可能性がある。

▷ deinit() が不要なモジュール

LedModule や ButtonModule は `pinMode()` で設定した GPIO を使うだけで、特別なリソースを占有しない。こうしたモジュールでは `deinit()` をオーバーライドしなくてよい。

6.4 API 仕様

BuzzerModule — ブザー出力モジュール

定義ファイル: `lib/BuzzerModule/BuzzerModule.h / .cpp`
分類: 出力モジュール（`updateOutput()` のみオーバーライド）
対象デバイス: パッシブブザー（PWM 駆動）
LEDC による PWM トーン再生を行うモジュール。周波数と再生時間を指定でき、時間経過後に自動停止する。

6.4.1 機能概要

- LEDC PWM によるトーン生成（任意の周波数）
- 再生時間指定（`durationMs`）による自動停止
- 再生リクエストフラグ（`requestPlay`）による制御
- `deinit()` による PWM チャネル解放

6.4.2 Config 構造体

◆ BuzzerConfig

ブザーの接続ピンと PWM チャネルを定義する。

メンバ	型	制約・既定値	説明
pin	uint8_t	—	ブザー出力 GPIO ピン番号
pwmChannel	uint8_t	0–15	LEDC チャネル番号。他モジュールと重複しないこと

6.4.3 Data 構造体

◆ BuzzerData

ブザーの再生パラメータを保持する。ロジックフェーズで設定し、出力フェーズで消費される。

メンバ	型	初期値	説明
frequency	uint16_t	0	再生周波数 [Hz]。0 で消音
durationMs	unsigned long	0	再生時間 [ms]。0 で無制限（停止は frequency = 0 で行う）
requestPlay	bool	false	再生リクエストフラグ。ロジックフェーズで true にセットする

6.4.4 メソッド

◆ BuzzerModule(const BuzzerConfig& config)

項目	内容
引数	config — BuzzerConfig への const 参照
動作	Config を_config にコピー保持する

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	LEDC チャネルの初期設定を行い、ブザーを消音状態にする

◆ void updateOutput(SystemData& data)

項目	内容
読み取り元	data.buzzer
タイミング制御	毎ループ実行
動作	requestPlay が true の場合、指定周波数で PWM トーンを開始し、durationMs > 0 なら時間計測を開始。requestPlay を false にクリア。再生中に durationMs 経過で自動停止

◆ void deinit()

項目	内容
動作	PWM トーンを停止し、LEDC チャネルをデタッチする

▷ requestPlay のライフサイクル

requestPlay はロジックフェーズで true にセットし、updateOutput() 内で消費（false にクリア）される。毎ループ true にし続けると再生が毎ループリスタートされるため、単発再生の場合は 1 回だけセットすること。

6.4.5 使用例

```
// ProjectConfig.h
const BuzzerConfig BUZZER_CONFIG = {
    .pin        = 47,  // GPIO47: ブザー出力
    .pwmChannel = 6,   // LEDCチャネル6
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // ボタン押下で「ピッ」と鳴らす
    if (data.button.justPressed) {
        data.buzzer.frequency   = 1000;  // 1kHz
        data.buzzer.durationMs  = 100;   // 100ms
        data.buzzer.requestPlay = true;
    }
}
```

Listing 18: BuzzerModule の使用例

7 【中級】Mpu6500Module — I2C バスパターン

Mpu6500Module で学べるパターン

- 通信バスパターン（コンストラクタでバスポインタを受け取る）
- ヘッダーでのバスクラス前方宣言（<Wire.h>依存を.cpp に閉じ込める）
- WHO_AM_I による init() でのデバイス識別
- 周期サンプリング + バースト読み取り

7.1 バスポインタの受け取り

```
// TwoWireはポインタのみ使用するため前方宣言で十分
class TwoWire;

class Mpu6500Module : public IModule {
private:
    Mpu6500Config _config;
    TwoWire*      _wire;        // コンストラクタで受け取るバスポインタ
    ModuleTimer   _sampleTimer;
public:
    explicit Mpu6500Module(const Mpu6500Config& config, TwoWire* wire);
    // ...
};
```

Listing 19: Mpu6500Module.h — バスの前方宣言とコンストラクタ

✓ ヘッダーでの前方宣言

class TwoWire; と前方宣言することで、ヘッダーに<Wire.h>をインクルードする必要がない。ポインタや参照の宣言には前方宣言だけで十分。これにより、Mpu6500Module.h をインクルードするファイルに Wire.h の依存が波及しない。

✓ Config にバスポインタ・バスピンを含めない理由

Mpu6500Config には I2C アドレスとサンプリング設定のみを含め、TwoWire*やバスピンは含めない。バスポインタを Config に含めると、ProjectConfig.h でバスインスタンスの extern 宣言が必要になり、インクルード順序の制約が生じる。I2C SDA/SCL のような共有バスピンは ProjectConfig.h に constexpr 単体定数として定義し、setup() 内の bus.begin() で直接参照する。コンストラクタの別引数にすることで、Config は純粋な設定値のみを持つ。

7.2 main.cpp でのバス管理

```
// バスインスタンス（グローバルスコープで生成）
static TwoWire mpuWire = TwoWire(0);

// モジュール生成（Config + バスポインタ）
Mpu6500Module mpu6500Module(MPU6500_CONFIG, &mpuWire);

```

```
void setup() {
    // バス初期化は全モジュールのinit()より前に実行
    mpuWire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    // ...
    mpu6500Module.init();  // init()ではbus.begin()を呼ばない
}
```

Listing 20: main.cpp — バスの生成と初期化

✓ init() で bus.begin() を呼ばない

バスの初期化は `main.cpp` の `setup()` で一元管理する。モジュールの `init()` ではデバイス固有の初期化（WHO_AM_I の確認、レジスタ設定等）のみ行う。同じバスを複数モジュールが共有する場合、各モジュールが `begin()` すると二重初期化になる。

7.3 API 仕様

Mpu6500Module — MPU-6500 IMU センサーモジュール

定義ファイル: `lib/Mpu6500Module/Mpu6500Module.h / .cpp`
分類: 入力モジュール（`updateInput()` のみオーバーライド）
対象デバイス: MPU-6500（I2C 接続、6 軸 IMU）
MPU-6500 から 3 軸加速度・3 軸ジャイロ・温度を取得するモジュール。ライブラリ非依存のレジスタ直接アクセスで実装されており、Wire によるバースト読み出しを行う。

7.3.1 機能概要

- I2C バースト読み出しによるセンサーデータ取得（14 バイト一括）
- 加速度 ±2g / ジャイロ ±250°/s のフルスケール
- 周期的サンプリング（`ModuleTimer` ベース）
- WHO_AM_I レジスタによるデバイス検証（`init()` 内）
- 外部から I2C バスインスタンス（`TwoWire*`）を注入する設計

7.3.2 Config 構造体

◆ `Mpu6500Config`

I2C 接続とサンプリングのパラメータを定義する。

メンバ	型	制約・既定値	説明
`address`	`uint8_t`	0x68 / 0x69	I2C スレーブアドレス。AD0=LOW: `0x68`、AD0=HIGH: `0x69`
`sampleIntervalMs`	`uint32_t`	例: 20	サンプリング周期 [ms]

▷ I2C バス注入パターン

I2C バスインスタンス（`TwoWire*`）は Config 構造体には含めず、コンストラクタの引数で渡す。バスの初期化（`Wire.begin(sda, scl)`）は `main.cpp` の `setup()` 内で行い、モジュールの `init()` ではバス初期化を呼ばない。共有バスピン（SDA/SCL）は `ProjectConfig.h` に `constexpr` 単体定数として定義し、`setup()` 内の `bus.begin()` で直接参照する。

7.3.3 Data 構造体

◆ `Mpu6500Data`

IMU センサーの計測値を保持する。すべての値は物理単位に変換済み。

メンバ	型	初期値	説明
`accelX`	`float`	`0.0f`	X 軸加速度 [g]
`accelY`	`float`	`0.0f`	Y 軸加速度 [g]
`accelZ`	`float`	`0.0f`	Z 軸加速度 [g]
`gyroX`	`float`	`0.0f`	X 軸角速度 [deg/s]
`gyroY`	`float`	`0.0f`	Y 軸角速度 [deg/s]
`gyroZ`	`float`	`0.0f`	Z 軸角速度 [deg/s]
`temperature`	`float`	`-999.0f`	チップ温度 [° C]
`isValid`	`bool`	`false`	WHO_AM_I 検証が成功し、データ読み取りが有効であれば `true`

7.3.4 メソッド

◆ `Mpu6500Module(const Mpu6500Config& config, TwoWire* wire)`

項目	内容
引数	`config` — `Mpu6500Config` への const 参照 `wire` — I2C バスインスタンスへのポインタ（`main.cpp` で生成）
動作	Config を `_config` にコピー、`wire` をメンバに保持

◆ `bool init()`

項目	内容
戻り値	`bool` — WHO_AM_I 検証成功で `true`、失敗で `false`
動作	WHO_AM_I レジスタ（`0x75`）を読み取り、期待値（`0x70`）と照合。一致すればスリープ解除、加速度・ジャイロのフルスケール設定を行う
失敗条件	WHO_AM_I の値が不一致（センサー未接続、アドレス誤り等）

◆ `void updateInput(SystemData& data)`

項目	内容
書き込み先	`data.mpu`
タイミング制御	`ModuleTimer` で `_config.sampleIntervalMs` ごとに実行
動作	レジスタ `0x3B` から 14 バイトをバースト読み出し。加速度（16384 LSB/g）・温度・ジャイロ（131 LSB/(deg/s)）を物理単位に変換して書き込む

▷ スケールファクタ

デフォルト設定（加速度 ±2g / ジャイロ ±250°/s）でのスケールファクタ:

軸	スケール	変換式
加速度	16384 LSB/g	$accel[g] = rawValue/16384.0$
ジャイロ	131 LSB/(deg/s)	$gyro[deg/s] = rawValue/131.0$
温度	—	$temp[°C] = rawValue/340.0 + 36.53$

7.3.5 使用例

```cpp
// ProjectConfig.h
constexpr int I2C_SDA_PIN = 41;
constexpr int I2C_SCL_PIN = 42;

const Mpu6500Config MPU6500_CONFIG = {
    .address          = 0x68,  // AD0=LOW
    .sampleIntervalMs = 20,    // 50Hz
};

// main.cpp
#include <Wire.h>
TwoWire i2cBus = TwoWire(0);
Mpu6500Module mpu(MPU6500_CONFIG, &i2cBus);

void setup() {
    i2cBus.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    mpu.init();
}

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    if (data.mpu.isValid) {
        float tilt = atan2(data.mpu.accelX, data.mpu.accelZ);
        // 傾きに応じた制御
    }
}
```

Listing 21: Mpu6500Module の使用例

8 【中級】DisplayBoardModule — 入出力統合 ＋ ライブラリ内部管理

DisplayBoardModule で学べるパターン

- 入出力統合モジュール（updateInput + updateOutput の両方をオーバーライド）
- 外部ライブラリ（TFT_eSPI）をモジュール内部にカプセル化する方法
- ModuleTimer による FPS 制御（更新頻度の制限）
- Data 構造体で入力データ（タッチ座標）と出力データ（表示テキスト）を統合管理

8.1 なぜ 1 モジュールに統合するのか

TFT ディスプレイボードは「タッチ入力」と「LCD 出力」の 2 機能を持つが、ハードウェア上は同じ SPI バスを共有する 1 枚の基板であり、同じドライバオブジェクト（TFT_eSPI）で管理される。このような場合、BleModule と同じく **1 つのモジュールで入出力の両方をオーバーライド**する方式が適切である。

✓ 入出力統合の判断基準

「入力と出力を別モジュールに分けるか、1 モジュールに統合するか」は、**ハードウェアの単位**で判断する。同一基板・同一ドライバで管理されるデバイスは統合し、物理的に独立したデバイスは別モジュールにする。BleModule の送受信統合と同じ考え方。

8.2 ライブラリの内部管理パターン

```
class TFT_eSPI;  // ヘッダへの依存を防ぐ

class DisplayBoardModule : public IModule {
private:
    DisplayBoardConfig _config;
    TFT_eSPI*          _tft;    // 内部で生成・管理
public:
    DisplayBoardModule(const DisplayBoardConfig& config);
    ~DisplayBoardModule();
    bool init()                          override;
    void updateInput(SystemData& data)   override;
    void updateOutput(SystemData& data)  override;
    void deinit()                        override;
};
```

Listing 22: DisplayBoardModule.h — TFT_eSPI の前方宣言

```
#include <TFT_eSPI.h>

DisplayBoardModule::DisplayBoardModule(const DisplayBoardConfig& config)
    : _config(config), _tft(new TFT_eSPI()) {}

DisplayBoardModule::~DisplayBoardModule() {
    delete _tft;
}
```

Listing 23: DisplayBoardModule.cpp — コンストラクタで TFT_eSPI を生成

✓ 外部バスパターンとの違い

Mpu6500Module は main.cpp で I2C バスを生成し、コンストラクタ引数で渡した（外部バスパターン）。DisplayBoardModule では TFT_eSPI が内部で SPI バスを管理するため、外部から渡す必要がない。SPI ピン設定は platformio.ini の build_flags でコンパイル時に決定される。

！既存ライブラリの制約

TFT_eSPI や LovyanGFX のように、ライブラリ内部でバスインスタンスを保持するライブラリでは、アーキテクチャの「外部からバスポインタを渡す」パターンをそのまま適用できない。この場合、ライブラリごとモジュール内部にカプセル化し、Config 構造体や build_flags で設定を渡す方式で対応する。

8.3 Data 構造体の入出力統合

```
struct DisplayBoardData {
    // 出力: 表示テキスト（ロジックフェーズが書き込む）
    char line1[48] = "";
    char line2[48] = "";
    char line3[48] = "";
    char line4[48] = "";
    char line5[48] = "";        // 黄色テキスト
    // 入力: タッチ状態（入力フェーズで更新）
    bool     touchPressed = false;
    uint16_t touchX       = 0; // ピクセル座標
    uint16_t touchY       = 0;
};
```

Listing 24: DisplayBoardModule.h — Data

✓ 入力と出力のデータを 1 つの Data 構造体に

DisplayBoardData はタッチ入力（touchPressed 等）と表示出力（line1 等）の両方を含む。入力フェーズでタッチ座標が書き込まれ、ロジックフェーズでそれを参照して表示テキストを設定し、出力フェーズで画面に描画する。BleModule の BleData（受信データ + 送信データ）と同じ構成。

8.4 ロジックフェーズでの表示内容設定

```
void applyPattern(SystemData& data) {
    // MPU-6500データを表示用にフォーマット
    if (data.mpu.isValid) {
        snprintf(data.display.line1, sizeof(data.display.line1),
                 "Ax:%.2f Ay:%.2f Az:%.2f",
                 data.mpu.accelX, data.mpu.accelY, data.mpu.accelZ);
    }
    // タッチ座標を表示用にフォーマット
    if (data.display.touchPressed) {
        snprintf(data.display.line5, sizeof(data.display.line5),
                 "Touch: %3d, %3d",
                 data.display.touchX, data.display.touchY);
    }
}
```

Listing 25: main.cpp — applyPattern() での表示設定

✓ 3 フェーズモデルのデータフロー

入力フェーズで DisplayBoardModule が `data.display.touchPressed/X/Y` に値を書き込み、ロジックフェーズで MPU データやタッチ座標を文字列に変換して `data.display.line1` 等に書き込み、出力フェーズで DisplayBoardModule が画面に描画する。入力→ロジック→出力の典型的なデータフローの実例。

8.5 API 仕様

DisplayBoardModule — TFT 表示ボードモジュール

定義ファイル: `lib/DisplayBoardModule/DisplayBoardModule.h / .cpp`
分類: 入出力モジュール（`updateInput()` + `updateOutput()`）
対象デバイス: ST7789 TFT LCD + XPT2046 タッチパネル
同一基板上の LCD 表示（出力）とタッチ読み取り（入力）を統合管理するモジュール。TFT_eSPI ライブラリを内部で保持し、外部からのドライバ注入は不要。

8.5.1 機能概要

- ST7789 TFT LCD への 5 行テキスト表示
- XPT2046 タッチパネルの座標読み取り
- 画面更新周期の制御（`ModuleTimer` ベース）
- バックライト PWM 制御
- TFT_eSPI インスタンスの内部管理（new/delete）
- `deinit()` によるリソース解放

8.5.2 Config 構造体

◆ `DisplayBoardConfig`

LCD 表示のパラメータを定義する。SPI/タッチのピン番号は `platformio.ini` の `build_flags` で設定する。

メンバ	型	制約・既定値	説明
`rotation`	`uint8_t`	0–3（1=横向き）	画面回転方向
`blPin`	`int8_t`	-1 で常時 ON	バックライト制御 GPIO ピン
`updateIntervalMs`	`uint32_t`	例: 100	画面更新周期 [ms]。例: 100ms = 10fps

▷ SPI/タッチのピン設定

TFT_eSPI は SPI バスを内部で管理するため、外部 `SPIClass` は使用しない。ピン番号（MOSI/MISO/SCK/CS/DC/RST/タッチ CS）は `platformio.ini` の `build_flags` で設定する:

```
build_flags =
    -DTFT_MOSI=11  -DTFT_MISO=13  -DTFT_SCLK=12
```

```
-DTFT_CS=10    -DTFT_DC=14    -DTFT_RST=15
-DTOUCH_CS=16
```

8.5.3 Data 構造体

◆ DisplayBoardData

LCD 表示内容とタッチ入力状態を保持する。

メンバ	型	初期値	説明
— 出力: 表示内容（ロジックフェーズで設定） —			
line1[48]	char[]	""	1 行目テキスト
line2[48]	char[]	""	2 行目テキスト
line3[48]	char[]	""	3 行目テキスト
line4[48]	char[]	""	4 行目テキスト
line5[48]	char[]	""	5 行目テキスト（タッチ座標表示等）
— 入力: タッチ状態（入力フェーズで更新） —			
touchPressed	bool	false	タッチ押下状態
touchX	uint16_t	0	タッチ X 座標 [px]
touchY	uint16_t	0	タッチ Y 座標 [px]

8.5.4 メソッド

◆ DisplayBoardModule(const DisplayBoardConfig& config)

項目	内容
引数	config — DisplayBoardConfig への const 参照
動作	Config を_config にコピー。TFT_eSPI インスタンスを new で生成

◆ ~DisplayBoardModule()

項目	内容
動作	内部の TFT_eSPI インスタンスを delete で解放する

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	TFT 初期化（tft.init()）、画面回転設定、背景色クリア、バックライト ON、タッチキャリブレーション

◆ void updateInput(SystemData& data)

項目	内容
書き込み先	data.display（タッチ関連メンバ）
タイミング制御	毎ループ実行
動作	タッチパネルの押下状態を取得し、touchPressed・touchX・touchY を更新

◆ void updateOutput(SystemData& data)

項目	内容
読み取り元	data.display（テキスト関連メンバ）
タイミング制御	ModuleTimer で updateIntervalMs ごとに実行
動作	画面をクリアし、line1–line5 のテキストを描画する

◆ void deinit()

項目	内容
動作	バックライト OFF、画面クリア

8.5.5 使用例

```
// ProjectConfig.h
const DisplayBoardConfig DISPLAY_BOARD_CONFIG = {
    .rotation        = 1,    // 横向き
    .blPin           = 46,   // バックライトピン
    .updateIntervalMs = 100, // 10fps
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // センサーデータを表示
    snprintf(data.display.line1, 48,
             "Accel: %.2f g", data.mpu.accelX);
    snprintf(data.display.line2, 48,
             "Battery: %.1f V", data.battery.voltage);
    // タッチ座標の表示
    if (data.display.touchPressed) {
        snprintf(data.display.line5, 48,
                 "Touch: (%d, %d)",
                 data.display.touchX,
                 data.display.touchY);
    }
}
```

Listing 26: DisplayBoardModule の使用例

9 【中級】CameraModule — PSRAM バッファ + フレーム管理

CameraModule で学べるパターン

- PSRAM への大容量バッファ配置
- フレームバッファのライフサイクル管理（取得→使用→解放）
- Data 構造体にメタデータのみ格納し、生データはモジュール内に保持
- 多数のピンを Config 構造体で一元管理

9.1 Config の大量ピン管理

```
struct CameraConfig {
    int8_t pwdnPin, resetPin;           // 電源/リセット
    int8_t xclkPin, siodPin, siocPin;   // クロック/SCCB
    int8_t d0Pin, d1Pin, d2Pin, d3Pin;  // DVPデータバス
    int8_t d4Pin, d5Pin, d6Pin, d7Pin;
    int8_t vsyncPin, hrefPin, pclkPin;  // 同期
    int     xclkFreqHz;
    uint8_t frameSize, pixFormat;
    int     jpegQuality;
    size_t  fbCount;
};
```

Listing 27: CameraModule.h — Config（多数のピン）

✓ Config 構造体の威力

カメラモジュールはピンが 16 本以上ある。コンストラクタの引数で 16 個のピンを渡すと順序を間違える危険があるが、Config 構造体なら`.d0Pin = 11` のように名前で指定できる。Config 構造体のピンアサインが `ProjectConfig.h` に集約されるため、回路設計者はこのファイルだけでピンの確認・変更ができる。

9.2 フレームバッファの管理

```
// updateInput()はメタデータだけをSystemDataに書き込む
void CameraModule::updateInput(SystemData& data) {
    // 保持中のフレームがあれば先に解放
    releaseFrame();

    camera_fb_t* fb = esp_camera_fb_get();
    if (fb) {
        data.camera.frameReady = true;
        data.camera.width = fb->width;
        data.camera.height = fb->height;
        data.camera.frameSize = fb->len;
        _frameBuffer = fb;  // void*で保持（ヘッダ依存回避）
        _frameHeld = true;
    }
}

```

```
// ロジックフェーズから生データにアクセスする場合
const uint8_t* CameraModule::getFrameBuffer() const {
    if (!_frameHeld || !_frameBuffer) return nullptr;
    return ((camera_fb_t*)_frameBuffer)->buf;
}
void CameraModule::releaseFrame() {
    if (_frameHeld && _frameBuffer) {
        esp_camera_fb_return((camera_fb_t*)_frameBuffer);
        _frameBuffer = nullptr;
        _frameHeld = false;
    }
}
```

Listing 28: CameraModule — フレームバッファ管理

✓ Data 構造体に生データを入れない

`camera_fb_t*`（フレームバッファポインタ）は SystemData に格納しない。SystemData に入れると、他モジュールがフレームの解放タイミングを制御できてしまう。代わりに `getFrameBuffer()` / `releaseFrame()` でアクセスを制御し、`CameraData` にはメタデータ（サイズ、解像度等）のみを格納する。

▷ PSRAM の自動配置

`fbCount >= 2` の場合、esp_camera ライブラリが自動的に PSRAM にバッファを配置する。Config 構造体の `fbCount` で制御でき、モジュール内部で PSRAM 判定のコードを書く必要はない。

9.3 API 仕様

CameraModule — OV5640 カメラモジュール

定義ファイル: `lib/CameraModule/CameraModule.h / .cpp`
分類: 入力モジュール（`updateInput()` のみオーバーライド）
対象デバイス: OV5640（ESP32-S3 DVP 接続）
ESP32-S3 の DVP インターフェース経由で OV5640 カメラからフレームを取得するモジュール。esp_camera ライブラリを使用し、フレームバッファの取得・解放を管理する。フレームの生データは `SystemData` には格納せず、専用メソッドでアクセスする。

9.3.1 機能概要

- DVP 16 ピンインターフェースによるカメラ接続
- JPEG/RAW フォーマット対応（`pixFormat` で選択）
- フレームバッファの取得・解放 API（`getFrameBuffer()` / `releaseFrame()`）
- フレームメタデータ（解像度・サイズ）の `SystemData` への書き込み
- `deinit()` によるカメラリソース解放

9.3.2 Config 構造体

◆ CameraConfig

DVP ピンアサインとカメラ動作パラメータを定義する。

メンバ	型	制約・既定値	説明
— DVP ピンアサイン —			
pwdnPin	int8_t	-1 で未使用	POWER DOWN ピン
resetPin	int8_t	-1 で未使用	ハードウェア RESET ピン
xclkPin	int8_t	—	XCLK クロック出力ピン
siodPin	int8_t	—	SCCB SDA（カメラ設定用 I2C）
siocPin	int8_t	—	SCCB SCL
d0Pin–d7Pin	int8_t	—	データバス D0–D7（8 ピン）
vsyncPin	int8_t	—	垂直同期ピン
hrefPin	int8_t	—	水平参照ピン
pclkPin	int8_t	—	ピクセルクロックピン
— カメラ動作設定 —			
xclkFreqHz	int	通常: 20000000	XCLK クロック周波数 [Hz]
frameSize	uint8_t	例: 5 (QVGA)	フレームサイズ（framesize_t の値）
pixFormat	uint8_t	例: 4 (JPEG)	ピクセルフォーマット（pixformat_t の値）
jpegQuality	int	0–63（低い=高画質）	JPEG 圧縮品質
fbCount	size_t	推奨: 2	フレームバッファ数

9.3.3 Data 構造体

◆ CameraData

フレームのメタデータを保持する。フレームの生データ（ピクセルバッファ）は含まない。

メンバ	型	初期値	説明
isValid	bool	false	カメラ初期化が成功していれば true
frameReady	bool	false	新しいフレームが取得済みであれば true
width	uint16_t	0	フレーム幅 [px]
height	uint16_t	0	フレーム高さ [px]
frameSize	size_t	0	フレームデータサイズ [bytes]

！ フレームデータのアクセス方法

フレームの生データ（ピクセルバッファ）は大容量のため SystemData には格納しない。ロジックフェーズで `getFrameBuffer()` を呼び出してポインタを取得し、使用後は必ず `releaseFrame()` で解放すること。解放しないとフレームバッファが枯渇し、次のフレーム取得がブロックされる。

9.3.4 メソッド

◆ `CameraModule(const CameraConfig& config)`

項目	内容
引数	`config` — `CameraConfig` への const 参照
動作	Config を `_config` にコピー保持する

◆ `bool init()`

項目	内容
戻り値	`bool` — カメラ初期化成功で `true`、失敗で `false`
動作	Config から esp_camera の設定構造体を構築し、`esp_camera_init()` を呼び出す
失敗条件	カメラ未接続、ピン設定誤り、PSRAM メモリ不足等

◆ `void updateInput(SystemData& data)`

項目	内容
書き込み先	`data.camera`
タイミング制御	毎ループ実行
動作	前回のフレームが保持中であれば解放し、新しいフレームを `esp_camera_fb_get()` で取得。メタデータ（幅、高さ、サイズ）を `data.camera` に書き込む

◆ `const uint8_t* getFrameBuffer() const`

項目	内容
戻り値	フレームデータへのポインタ。フレーム未取得時は `nullptr`
用途	ロジックフェーズでフレームデータにアクセスする（BLE 送信、SD 保存、画像処理等）
有効期間	次の `updateInput()` 呼び出しまで、または `releaseFrame()` 呼び出しまで

◆ `void releaseFrame()`

項目	内容
動作	保持中のフレームバッファを `esp_camera_fb_return()` で解放する。フレーム未保持時は何もしない

◆ void deinit()

項目	内容
動作	保持中のフレームを解放し、esp_camera_deinit() でカメラリソースを解放する

！ PSRAM の必要性

fbCount を 2 以上に設定する場合、PSRAM が必要である。PSRAM がない環境では fbCount = 1 に制限し、フレームサイズも小さくする必要がある。platformio.ini で board_build.psram = enabled を設定すること。

9.3.5 使用例

```
// ProjectConfig.h
const CameraConfig CAMERA_CONFIG = {
    .pwdnPin = -1, .resetPin = -1, .xclkPin = 15,
    .siodPin = 4,  .siocPin = 5,
    .d0Pin = 11, .d1Pin = 9, .d2Pin = 8, .d3Pin = 10,
    .d4Pin = 12, .d5Pin = 18, .d6Pin = 17, .d7Pin = 16,
    .vsyncPin = 6, .hrefPin = 7, .pclkPin = 13,
    .xclkFreqHz  = 20000000,
    .frameSize   = 5,   // QVGA (320x240)
    .pixFormat   = 4,   // JPEG
    .jpegQuality = 12,
    .fbCount     = 2,   // ダブルバッファ (PSRAM必須)
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    if (data.camera.frameReady) {
        const uint8_t* buf = camera.getFrameBuffer();
        size_t len = data.camera.frameSize;
        // BLE送信やSD保存などの処理
        camera.releaseFrame();
    }
}
```

Listing 29: CameraModule の使用例

10 【応用】DriveMotorModule — PWM 2 ピン制御 + 統合部品

DriveMotorModule で学べるパターン

- H ブリッジによる方向制御（IN1/IN2 + PWM）
- 逆転保護（ブレーキ自動挿入）
- drive() メソッドによる SystemData 非経由の直接制御
- 統合モジュールの「内部部品」としての使い方

10.1 2 つの使い方

DriveMotorModule は 2 つの用途を想定した設計になっている。

1. **統合モジュールの内部部品**（主な用途） — ChassisModule 等が内部で drive() を呼ぶ
2. **単体使用**（拡張時） — updateOutput() を実装し、outputModules[] に登録して使う（現在のコードでは空実装）

```
class DriveMotorModule : public IModule {
public:
    // SystemData経由の制御（空実装。単体使用時は書き換えが必要）
    void updateOutput(SystemData& data) override;

    // SystemData非経由の直接制御（統合モジュールから使用）
    void drive(float power);
};
```

Listing 30: DriveMotorModule.h — drive() メソッド

✓ updateOutput() と drive() の使い分け

現在の実装では updateOutput() は空実装であり、ChassisModule が drive() を直接呼ぶ設計。単体モーターとして outputModules[] に登録して使いたい場合は、updateOutput() 内で data.driveMotor.power を読み取り drive() を呼ぶように実装する。

10.2 逆転保護

```
void DriveMotorModule::_applyMotor(float power) {
    // 現在の方向を判定
    int8_t direction = 0;
    if (power > _config.minPowerThreshold) {
        direction = 1;   // 正転
    } else if (power < -_config.minPowerThreshold) {
        direction = -1;  // 逆転
    }

    // 正転<->逆転の直接切り替え時、1フレーム分のブレーキを挿入
    if ((direction == 1 && _prevDirection == -1) ||
        (direction == -1 && _prevDirection == 1)) {
        // ブレーキ（IN1=HIGH, IN2=HIGH で短絡ブレーキ）
        digitalWrite(_config.in1Pin, HIGH);
        digitalWrite(_config.in2Pin, HIGH);
        ledcWrite(_config.pwmChannel, MOTOR_DUTY_MAX);
        _prevDirection = 0;
        return;  // 次のupdateOutput()で実際の方向転換が行われる
    }
    // ... IN1/IN2方向設定 + PWMデューティ設定 ...
}
```

Listing 31: DriveMotorModule.cpp — 逆転保護

✓ 1 フレーム遅延によるモーター保護

正転→逆転の直接切り替えはモーターやドライバ IC に大きな負荷がかかる。1 フレーム分（1 ループ分）のブレーキを挿入することで、物理的に安全な方向転換を実現する。3 フェーズモデルの「毎ループ 1 回だけ呼ばれる」性質を利用した保護パターン。

10.3 API 仕様

DriveMotorModule — DC モーター単体制御モジュール

定義ファイル: `lib/DriveMotorModule/DriveMotorModule.h / .cpp`
分類: 出力モジュール（`updateOutput()` のみオーバーライド）
対象デバイス: DC モーター（H ブリッジドライバ経由）
IN1/IN2 で方向制御、PWM で速度制御を行う H ブリッジドライバ用モジュール。ChassisModule 等の統合モジュールが内部で保持し、`drive()` メソッドで直接制御する設計。

! `outputModules[]` に登録しない

DriveMotorModule は **ChassisModule 等の統合モジュールが内部で保持する**前提の設計である。`outputModules[]` には登録せず、ChassisModule の `updateOutput()` 経由で `drive()` が呼ばれる。`updateOutput()` は空実装であり、直接呼び出しても何も起きない。

10.3.1 機能概要

- IN1/IN2 + PWM による H ブリッジ制御（正転・逆転・停止）
- 出力値 −100~100% の範囲指定
- 最小出力閾値（`minPowerThreshold`）以下で停止扱い
- 方向反転時の 1 フレームブレーキ挿入（モーター保護）
- 外部直接制御用の `drive()` メソッド

10.3.2 Config 構造体

◆ `DriveMotorConfig`

H ブリッジドライバの接続ピンと PWM パラメータを定義する。

メンバ	型	制約・既定値	説明
`in1Pin`	`uint8_t`	—	方向制御ピン 1
`in2Pin`	`uint8_t`	—	方向制御ピン 2
`pwmPin`	`uint8_t`	—	PWM 速度制御ピン
`pwmChannel`	`uint8_t`	0–15	LEDC チャネル番号
`pwmFreqHz`	`uint32_t`	例: 1220	PWM 周波数 [Hz]
`minPowerThreshold`	`float`	例: 5.0	最小出力閾値 [%]。この値以下の出力は停止扱い

10.3.3 Data 構造体

◆ `DriveMotorData`

モーターの目標出力を保持する。

メンバ	型	初期値	説明
`power`	`float`	`0.0f`	目標出力 [−100.0~100.0]。正: 正転、負: 逆転、0: 停止

10.3.4 メソッド

◆ `DriveMotorModule(const DriveMotorConfig& config)`

項目	内容
引数	`config` — `DriveMotorConfig` への const 参照
動作	Config を `_config` にコピー保持する

◆ `bool init()`

項目	内容
戻り値	`bool` — 常に `true`
動作	IN1/IN2 ピンを `OUTPUT` に設定。PWM ピンの LEDC チャネルを初期化。モーターを停止状態にする

◆ `void updateOutput(SystemData& data)`

項目	内容
動作	空実装。ChassisModule が `drive()` を直接呼ぶため、このメソッドは使用しない

◆ `void drive(float power)`

項目	内容
引数	`power` — 出力値 [−100.0~100.0]
動作	出力値の絶対値が `minPowerThreshold` 以下なら停止。方向が前回と反転した場合は 1 フレームブレーキを挿入。IN1/IN2 で方向設定、PWM デューティで速度制御
呼び出し元	ChassisModule の `updateOutput()` 内

▷ 方向反転時のブレーキ

前回の方向と今回の方向が逆転した場合（正転→逆転、逆転→正転）、モーターと駆動回路を保護するため 1 フレーム分のブレーキ（IN1=IN2=HIGH、短絡ブレーキ）を挿入する。次のループで新しい方向の駆動が開始される。

10.3.5 使用例

```
// ChassisConfigのmotors配列で定義
const ChassisConfig CHASSIS_CONFIG = {
    .motors = {
        // [0] 左前
        {.in1Pin = 4,  .in2Pin = 16, .pwmPin = 17,
         .pwmChannel = 2, .pwmFreqHz = 1220,
         .minPowerThreshold = 5.0f},
        // ...（右前、左後、右後）
    },
    .wheelPattern     = WheelPattern::OMNI,
    .maxOutputPercent = 90.0f,
};
// DriveMotorModuleはChassisModuleが内部で生成・管理する
```

Listing 32: DriveMotorModule の使用例（ChassisModule 内部）

11 【応用】ChassisModule — 機能統合パターン

ChassisModule で学べるパターン

- 複数モジュールを内部に保持する「機能統合系モジュール」
- 内部モジュールの init()/updateOutput()/deinit() を親が管理
- ベクトル合成による高レベル制御（3 軸入力→ 4 輪出力）
- 内部モジュールの SystemData 登録が不要なケース

11.1 内部モジュールの保持

```
static constexpr int CHASSIS_MOTOR_COUNT = 4;

struct ChassisConfig {
    DriveMotorConfig motors[CHASSIS_MOTOR_COUNT];
    WheelPattern     wheelPattern;
    float            maxOutputPercent;
};

class ChassisModule : public IModule {
private:
    ChassisConfig    _config;
    DriveMotorModule _motorFL, _motorFR, _motorRL, _motorRR;
    DriveMotorModule* _motors[CHASSIS_MOTOR_COUNT];
public:
    ChassisModule(const ChassisConfig& config);
    bool init() override;
    void updateOutput(SystemData& data) override;
    void deinit() override;
};
```

Listing 33: ChassisModule.h — 内部に DriveMotorModule を 4 つ保持

✓ 内部モジュールは IModule 配列に登録しない

4 つの DriveMotorModule は ChassisModule のメンバ変数であり、main.cpp の outputModules[] には登録しない。SystemData にも DriveMotorData を含める必要がない。外部からは ChassisModule1 つだけが見え、3 軸入力だけで 4 輪を制御できる。

11.2 親モジュールからの制御委譲

```
ChassisModule::ChassisModule(const ChassisConfig& config)
    : _config(config),
      _motorFL(config.motors[0]),
      _motorFR(config.motors[1]),
      _motorRL(config.motors[2]),
      _motorRR(config.motors[3])
{
    _motors[0] = &_motorFL;
    _motors[1] = &_motorFR;
    _motors[2] = &_motorRL;
    _motors[3] = &_motorRR;
}
```

Listing 34: ChassisModule.cpp — init()/deinit() で内部モジュールを管理

▷ コンストラクタの初期化リスト — 初学者向け解説

コロン（:）の後に書かれている部分が「初期化リスト」です。ここでは以下の処理を行っています。

1. _config(config) — Config 構造体をコピー保存
2. _motorFL(config.motors[0]) — 各モーターを対応する Config で初期化

初期化リスト構文に慣れないうちは「コンストラクタ引数を受け取って、メンバ変数に渡している」とだけ理解すれば十分です。コンストラクタ本体では、ポインタ配列_motors[] に各モーターのアドレスをセットしています。これにより、ループで全モーターに対して統一的に操作できます。

```

bool ChassisModule::init() {
    for (int i = 0; i < CHASSIS_MOTOR_COUNT; i++) {
        if (!_motors[i]->init()) return false;
    }
    return true;
}

void ChassisModule::deinit() {
    for (int i = 0; i < CHASSIS_MOTOR_COUNT; i++) {
        _motors[i]->drive(0.0f);
    }
}
```

Listing 35: ChassisModule.cpp — init()/deinit()（続き）

```
void ChassisModule::updateOutput(SystemData& data) {
    float speeds[3] = {
```

```
        data.chassis.forwardSpeed,
        data.chassis.lateralSpeed,
        data.chassis.rotationSpeed,
    };
    // ベクトル合成: 方向行列 × 入力速度 → 各モーター出力
    float motorPowers[CHASSIS_MOTOR_COUNT] = {};
    const float (*matrix)[CHASSIS_MOTOR_COUNT] = _getMatrix();
    for (int dir = 0; dir < 3; dir++) {
        for (int m = 0; m < CHASSIS_MOTOR_COUNT; m++) {
            motorPowers[m] += matrix[dir][m] * speeds[dir];
        }
    }
    // リミッタ: 最大絶対値が100を超えたら全体を等比縮小
    float maxAbs = 0.0f;
    for (int m = 0; m < CHASSIS_MOTOR_COUNT; m++) {
        float absVal = fabsf(motorPowers[m]);
        if (absVal > maxAbs) maxAbs = absVal;
    }
    if (maxAbs > 100.0f) {
        float scale = 100.0f / maxAbs;
        for (int m = 0; m < CHASSIS_MOTOR_COUNT; m++) {
            motorPowers[m] *= scale;
        }
    }
    // 最大出力割合でスケーリング & 各モーター駆動
    float outputScale = _config.maxOutputPercent / 100.0f;
    for (int m = 0; m < CHASSIS_MOTOR_COUNT; m++) {
        motorPowers[m] *= outputScale;
        _motors[m]->drive(motorPowers[m]);
        data.chassis.motorOutputs[m] = motorPowers[m];
    }
}
```

Listing 36: ChassisModule.cpp — updateOutput() でベクトル合成→各モーターを drive()

✓ 機能統合の原則

仕様書の「モジュール間の依存関係は存在しないものとして設計する」ルールに従い、依存関係があるモジュール同士は 1 つに統合する。ChassisModule と DriveMotorModule は明確に依存関係があるため、ChassisModule が DriveMotorModule を内部部品として包含している。

▷ Config 構造体のネスト

`ChassisConfig.motors[4]` は `DriveMotorConfig` の配列。Config 構造体は別の Config 構造体をメンバとして持てる。これにより `ProjectConfig.h` で 4 輪分のピンアサインを一括定義できる。

11.3 API 仕様

ChassisModule — 4 輪統合足回り制御モジュール

定義ファイル: `lib/ChassisModule/ChassisModule.h / .cpp`
分類: 機能統合モジュール（`updateOutput()` のみオーバーライド）
対象デバイス: 4 輪駆動足回り（オムニ/メカナム/4WD）
内部で 4 つの DriveMotorModule を保持し、前進・横移動・回転の 3 軸ベクトルをホイールパターンに応じた方向行列で各モーター出力に合成する統合モジュール。

11.3.1 機能概要

- 4 つの DriveMotorModule の内部管理（生成・init・drive 呼び出し）
- 3 種類のホイールパターン対応（OMNI / MECANUM / FOURWD）
- 3 軸ベクトル（前進・横移動・回転）から 4 輪出力への合成
- 出力リミッタ（`maxOutputPercent` で最大出力を制限）
- 各モーターの実出力値をデバッグ用に保持

◆ 定数

名前	値	説明
`CHASSIS_MOTOR_COUNT`	4	モーター数（固定）

11.3.2 列挙型

◆ `WheelPattern`

ホイール配置パターンを定義する。パターンによりベクトル合成の方向行列が異なる。

値	名前	説明
0	`OMNI`	オムニホイール。全方向移動 + 回転が可能
1	`MECANUM`	メカナムホイール。全方向移動 + 回転が可能（ローラー角度が異なる）
2	`FOURWD`	4 輪駆動。前後移動 + 回転のみ（横移動不可）

11.3.3 Config 構造体

◆ `ChassisConfig`

4 輪足回りの構成を定義する。

メンバ	型	制約・既定値	説明
`motors[4]`	`DriveMotorConfig[]`	—	各モーターの設定。[0]: 左前、[1]: 右前、[2]: 左後、[3]: 右後

wheelPattern	WheelPattern	—	ホイールパターン
maxOutputPercent	float	例: 90.0	最大出力割合 [%]。合成後の出力をこの値でスケーリング

11.3.4 Data 構造体

◆ ChassisData

足回りの制御入力とデバッグ用出力を保持する。

メンバ	型	初期値	説明
— 入力（ロジックフェーズで設定） —			
forwardSpeed	float	0.0f	前進速度 [−100~100]。正: 前進、負: 後退
lateralSpeed	float	0.0f	横移動速度 [−100~100]。正: 右、負: 左
rotationSpeed	float	0.0f	回転速度 [−100~100]。正: 右回転、負: 左回転
— 出力（updateOutput 後に更新、デバッグ用） —			
motorOutputs[4]	float[]	{0}	各モーターの実出力値 [%]。[0]: 左前、[1]: 右前、[2]: 左後、[3]: 右後

11.3.5 メソッド

◆ ChassisModule(const ChassisConfig& config)

項目	内容
引数	config — ChassisConfig への const 参照
動作	Config を_config にコピー。4 つの DriveMotorModule を config.motors[i] から内部生成し、ポインタ配列_motors[] にセット

◆ bool init()

項目	内容
戻り値	bool — 4 モーター全ての init() が成功で true、1 つでも失敗で false
動作	内部の 4 つの DriveMotorModule の init() を順次呼び出す

◆ `void updateOutput(SystemData& data)`

項目	内容
読み取り元	`data.chassis`
タイミング制御	毎ループ実行
動作	ホイールパターンに応じた方向行列を取得し、`forwardSpeed`・`lateralSpeed`・`rotationSpeed` を合成して各モーター出力を計算。出力リミッタ適用後、各 DriveMotorModule の `drive()` を呼び出す。`motorOutputs[]` にデバッグ値を書き込む

◆ `void deinit()`

項目	内容
動作	全モーターを停止（`drive(0)`）させる

▷ ベクトル合成と方向行列

各モーターの出力は以下の式で計算される:

$$output_m = M_{0,m} \times forward + M_{1,m} \times lateral + M_{2,m} \times rotation$$

ここで M はホイールパターンに応じた 3×4 の方向行列であり、行が方向（前進・横移動・回転）、列がモーター（左前・右前・左後・右後）を表す。

OMNI（オムニホイール）の方向行列:

	前進	横移動	回転
左前 [0]	-1	$+1$	-1
右前 [1]	$+1$	$+1$	-1
左後 [2]	-1	-1	-1
右後 [3]	$+1$	-1	-1

合成後の出力は絶対値の最大値が 100 を超える場合、全体を正規化して ±100 以内に収める。その後、`maxOutputPercent` でスケーリングする。

！ DriveMotorModule の管理

ChassisModule が内部で 4 つの DriveMotorModule インスタンスを保持し、`init()`・`drive()` を呼び出す。個別の DriveMotorModule を `outputModules[]` に登録してはならない（二重制御になる）。

11.3.6 使用例

```
// ProjectConfig.h
const ChassisConfig CHASSIS_CONFIG = {
    .motors = {
        {.in1Pin=4, .in2Pin=16, .pwmPin=17,
         .pwmChannel=2, .pwmFreqHz=1220,
         .minPowerThreshold=5.0f},  // 左前
        {.in1Pin=26, .in2Pin=25, .pwmPin=33,
```

```
8           .pwmChannel=3, .pwmFreqHz=1220,
9           .minPowerThreshold=5.0f},  // 右前
10         {.in1Pin=18, .in2Pin=19, .pwmPin=23,
11          .pwmChannel=4, .pwmFreqHz=1220,
12          .minPowerThreshold=5.0f},  // 左後
13         {.in1Pin=13, .in2Pin=14, .pwmPin=27,
14          .pwmChannel=5, .pwmFreqHz=1220,
15          .minPowerThreshold=5.0f},  // 右後
16     },
17     .wheelPattern     = WheelPattern::OMNI,
18     .maxOutputPercent = 90.0f,
19 };
20
21 // ロジックフェーズでの使用
22 void applyPattern(SystemData& data) {
23     // BLE受信データから操作指令を反映
24     if (data.ble.newDataIn) {
25         data.chassis.forwardSpeed  = (int8_t)data.ble.rxData[0];
26         data.chassis.lateralSpeed  = (int8_t)data.ble.rxData[1];
27         data.chassis.rotationSpeed = (int8_t)data.ble.rxData[2];
28         data.ble.newDataIn = false;
29     }
30 }
```

Listing 37: ChassisModule の使用例

12 【応用】BleModule — スレッドセーフティパターン

BleModule で学べるパターン

- volatile フラグ方式による安全なデータ受け渡し
- BLE コールバック（別タスク）→内部バッファ→ updateInput() → SystemData
- deinit() による BLE リソースの解放
- 双方向通信（RX: updateInput() で受信反映、TX: updateOutput() で送信）

12.1 内部バッファと volatile フラグ

```
1 class BleModule : public IModule {
2 private:
3     // 内部バッファ（BLEコールバックスレッドが書き込む）
4     uint8_t       _rxBuffer[BLE_RX_BUFFER_SIZE] = {};
5     uint8_t       _rxBufferLength = 0;
6     volatile bool _dataReceived   = false;  // volatileフラグ
7     volatile bool _clientConnected = false;
8 public:
9     // BLEコールバックから呼ばれる（別タスクで実行される）
10    void onReceive(const uint8_t* payload, uint8_t length);
11    // ...
12 };
```

Listing 38: BleModule.h — 内部バッファ

！ なぜ volatile が必要か

BLE コールバックは Arduino のメインループとは別の FreeRTOS タスクで実行される。volatile がないと、コンパイラが_dataReceived の値をレジスタにキャッシュし、メインループ側で変更を検出できない場合がある。volatile は「この変数は外部から変更されるかもしれない」とコンパイラに伝えるキーワード。

12.2 コールバック→内部バッファ→ SystemData

```
// BLEコールバック（別タスクから呼ばれる）
void BleModule::onReceive(const uint8_t* payload, uint8_t length) {
    memcpy(_rxBuffer, payload, length);  // (1) バッファに書き込み
    _rxBufferLength = length;
    _dataReceived = true;                // (2) フラグを立てる
}

// updateInput()は入力フェーズで呼ばれる（メインタスク）
void BleModule::updateInput(SystemData& data) {
    data.ble.connected = _clientConnected;

    if (_dataReceived) {                            // (3) フラグ確認
        memcpy(data.ble.rxData, _rxBuffer, _rxBufferLength); // (4) コピー
        data.ble.rxLength  = _rxBufferLength;
        data.ble.newDataIn = true;
        _dataReceived = false;                      // (5) フラグクリア
    }
}

// updateOutput()は出力フェーズで呼ばれる
void BleModule::updateOutput(SystemData& data) {
    // 送信リクエスト処理
    if (data.ble.sendRequest && _clientConnected && _txChar) {
        _txChar->setValue(data.ble.txData, data.ble.txLength);
        _txChar->notify();
        data.ble.sendRequest = false;
    }
}
```

Listing 39: BleModule.cpp — データ受け渡しの流れ

✓ volatile フラグ方式の安全性

bool 型の読み書きは原子的（1 命令で完了する）。「バッファに全データを書き込んだ後にフラグを立てる」→「フラグを確認してからバッファを読む」の順序により、バッファの途中状態を読むことがない。mutex やセマフォは不要で、割り込みコンテキストでも安全に使える。

12.3 BLE コールバックの登録

```
// BLEライブラリのコールバッククラスを継承
class BleRxCallbacks : public BLECharacteristicCallbacks {
private:
    BleModule* _module;
public:
    BleRxCallbacks(BleModule* module) : _module(module) {}
    void onWrite(BLECharacteristic* characteristic) override {
        std::string value = characteristic->getValue();
        uint8_t len = min((int)value.length(), BLE_RX_BUFFER_SIZE);
        _module->onReceive(
            reinterpret_cast<const uint8_t*>(value.data()), len);
    }
};
```

Listing 40: BleModule.cpp — コールバッククラス

✓ コールバックからモジュールへの橋渡し

BLE ライブラリはコールバッククラスの継承を要求するため、内部クラスでラップして `BleModule::onReceive()` に橋渡しする。コールバッククラスはモジュールのポインタを保持し、受信データをモジュールの内部バッファに渡す。

12.4 送信の流れ（ロジックフェーズ→ updateOutput()）

```
void applyPattern(SystemData& data) {
    // BLE受信データの処理
    if (data.ble.newDataIn) {
        // 受信データを処理...
        data.ble.newDataIn = false;  // 処理済みフラグクリア
    }

    // BLE送信（応答を返す例）
    if (needToRespond) {
        memcpy(data.ble.txData, response, responseLen);
        data.ble.txLength = responseLen;
        data.ble.sendRequest = true;  // 次のupdateOutput()で送信される
    }
}
```

Listing 41: ロジックフェーズでの送信リクエスト

▷ 双方向通信のデータフロー

受信: BLE コールバック → 内部バッファ → updateInput() → SystemData.ble.rxData → ロジックフェーズ

送信: ロジックフェーズ → SystemData.ble.txData → updateOutput() → BLE Notify

BleModule は `inputModules[]` と `outputModules[]` の両方に登録する。受信処理は入力フェーズの `updateInput()` で、送信処理は出力フェーズの `updateOutput()` で実行される。

12.5 API 仕様

BleModule — BLE 通信モジュール

定義ファイル: `lib/BleModule/BleModule.h / .cpp`
分類: 入出力モジュール（`updateInput()` + `updateOutput()`）
対象デバイス: ESP32 BLE（GATT Server）
BLE GATT Server として動作し、RX/TX Characteristic を使用してデータの受信・送信を行う。BLE コールバックは別タスクで実行されるため、volatile フラグ方式でスレッドセーフにデータを受け渡す。

！ スレッドセーフティ

BLE のコールバック（`onConnect`、`onDisconnect`、`onReceive`）は FreeRTOS の別タスクで実行される。コールバックから `SystemData` に直接書き込むことは禁止。内部バッファ + `volatile` フラグ方式で `updateInput()` を経由してデータを受け渡す。

12.5.1 機能概要

- BLE GATT Server の初期化・アドバタイズ開始
- RX Characteristic 経由のデータ受信（コールバック→内部バッファ→ SystemData）
- TX Characteristic 経由のデータ送信（SystemData → notify）
- 接続状態の監視と切断時の自動リアドバタイズ
- `deinit()` による BLE リソース解放

◆ 定数

名前	値	説明
`BLE_RX_BUFFER_SIZE`	128	受信バッファサイズ [bytes]
`BLE_TX_BUFFER_SIZE`	128	送信バッファサイズ [bytes]

12.5.2 Config 構造体

◆ `BleConfig`

BLE サービスの識別情報を定義する。

メンバ	型	制約・既定値	説明
`deviceName`	`const char*`	—	BLE デバイス名（アドバタイズ時に表示）
`serviceUuid`	`const char*`	—	GATT サービス UUID 文字列
`rxCharUuid`	`const char*`	—	RX Characteristic UUID（クライアント→デバイス方向）
`txCharUuid`	`const char*`	—	TX Characteristic UUID（デバイス→クライアント方向）

12.5.3 Data 構造体

◆ BleData

BLE 通信の状態と送受信データを保持する。

メンバ	型	初期値	説明
— 接続状態 —			
connected	bool	false	BLE 接続状態
— 受信（入力フェーズで更新） —			
newDataIn	bool	false	新規受信データあり。ロジックフェーズで参照後クリアする
rxData[128]	uint8_t[]	{}	受信データバッファ
rxLength	uint8_t	0	受信データ長 [bytes]
— 送信（ロジックフェーズで設定） —			
sendRequest	bool	false	送信リクエスト。true で updateOutput() がデータを送信
txData[128]	uint8_t[]	{}	送信データバッファ
txLength	uint8_t	0	送信データ長 [bytes]

12.5.4 メソッド

◆ BleModule(const BleConfig& config)

項目	内容
引数	config — BleConfig への const 参照
動作	Config を_config にコピー保持する

◆ bool init()

項目	内容
戻り値	bool — サーバー・サービス作成成功で true、失敗で false
動作	BLE デバイス初期化、GATT サーバー作成、サービス・Characteristic 登録、コールバック設定、アドバタイズ開始
失敗条件	BLEDevice::createServer() またはサービス作成の失敗

◆ `void updateInput(SystemData& data)`

項目	内容
書き込み先	`data.ble`
タイミング制御	毎ループ実行
動作	`volatile` フラグ `_dataReceived` を確認。`true` なら内部バッファから `data.ble.rxData` にコピーし、`newDataIn = true` に設定。接続状態を `data.ble.connected` に反映

◆ `void updateOutput(SystemData& data)`

項目	内容
読み取り元	`data.ble`
タイミング制御	毎ループ実行
動作	`sendRequest` が `true` かつ接続中であれば、`txData` を TX Characteristic に notify で送信。`sendRequest` を `false` にクリア

◆ `void deinit()`

項目	内容
動作	BLE アドバタイズを停止し、BLE デバイスを終了処理する

◆ `void onConnect() / void onDisconnect() / void onReceive(const uint8_t* payload, uint8_t length)`

項目	内容
呼び出し元	BLE コールバック（別タスクで実行される）
動作	`onConnect`: `_clientConnected = true`。`onDisconnect`: `_clientConnected = false`、リアドバタイズ開始。`onReceive`: 内部バッファにコピーし `_dataReceived = true`

▷ volatile フラグ方式のデータフロー

BLE コールバック（別タスク）から SystemData へのデータ受け渡しは以下の流れで行う:

1. **コールバック**: 受信データを内部バッファ `_rxBuffer` にコピーし、`volatile bool _dataReceived = true`
2. **updateInput()**: `_dataReceived` を確認。`true` なら内部バッファから `data.ble.rxData` にコピーし、`_dataReceived = false` にリセット
3. **ロジックフェーズ**: `data.ble.newDataIn` を確認して受信データを処理

12.5.5 使用例

```
// ProjectConfig.h
const BleConfig BLE_CONFIG = {
    .deviceName  = "ESP32-Module",
    .serviceUuid = "12345678-1234-1234-1234-123456789abc",
```

```
    .rxCharUuid  = "12345678-1234-1234-1234-123456789abd",
    .txCharUuid  = "12345678-1234-1234-1234-123456789abe",
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    // 受信データの処理
    if (data.ble.newDataIn) {
        // rxDataを解析
        data.ble.newDataIn = false;  // フラグクリア
    }
    // センサーデータのBLE送信
    if (data.ble.connected && data.mpu.isValid) {
        memcpy(data.ble.txData, &data.mpu.accelX,
               sizeof(float));
        data.ble.txLength    = sizeof(float);
        data.ble.sendRequest = true;
    }
}
```

Listing 42: BleModule の使用例

13 【応用】WifiModule — ステートマシンパターン

WifiModule で学べるパターン

- enum class による状態定義と switch 文による状態遷移
- ModuleTimer によるタイムアウト・再接続間隔の管理
- リクエストフラグによる外部からの制御（接続/切断指示）
- deinit() による WiFi リソースの解放

13.1 状態定義

```
enum class WifiState : uint8_t {
    DISCONNECTED,  // 未接続（接続リクエスト待ち）
    CONNECTING,    // 接続試行中
    CONNECTED,     // 接続済み
    RECONNECTING,  // 再接続試行中
    FAILED,        // 接続失敗（リトライ上限超過）
};
```

Listing 43: WifiModule.h — 状態定義

✓ Data 構造体にステートを公開

`WifiData.state` として enum 値を SystemData に公開することで、ロジックフェーズが現在の接続状態に応じた判断を行える。`isConnected` は簡易参照用の bool で、`state == CONNECTED` の省略形。

13.2 ステートマシンの実装

```
void WifiModule::updateInput(SystemData& data) {
    // ステートマシン（WiFi状態の監視・自動再接続）
    switch (_state) {
        case WifiState::CONNECTING:
        case WifiState::RECONNECTING:
            if (WiFi.status() == WL_CONNECTED) {
                _state = WifiState::CONNECTED;
                _retryCount = 0;
            } else if (_stateTimer.getNowTime()
                        >= _config.connectTimeoutMs) {
                WiFi.disconnect();
                _retryCount++;
                if (_config.maxRetries > 0
                    && _retryCount >= _config.maxRetries) {
                    _state = WifiState::FAILED;
                } else {
                    _state = WifiState::RECONNECTING;
                    _stateTimer.setTime();
                }
            }
            break;

        case WifiState::CONNECTED:
            if (WiFi.status() != WL_CONNECTED) {
                _state = WifiState::RECONNECTING;
                _stateTimer.setTime();
            }
            break;
        // ...
    }

    // SystemDataに状態を反映
    data.wifi.state       = _state;
    data.wifi.isConnected = (_state == WifiState::CONNECTED);
    data.wifi.retryCount  = _retryCount;
}
```

Listing 44: WifiModule.cpp — updateInput() のステートマシン

```
void WifiModule::updateOutput(SystemData& data) {
    // ロジックフェーズからのリクエスト処理
    if (data.wifi.requestConnect) {
        data.wifi.requestConnect = false;
        if (_state == WifiState::DISCONNECTED
            || _state == WifiState::FAILED) {
            _retryCount = 0;
            _startConnect();
        }
    }
    if (data.wifi.requestDisconnect) {
        data.wifi.requestDisconnect = false;
        _disconnect();
```

```
    }
}
```

Listing 45: WifiModule.cpp — updateOutput() のリクエスト処理

✓ ノンブロッキングなステートマシン

WiFi 接続は通常数秒かかるが、`delay()` で待たない。`CONNECTING` 状態では毎ループ `WiFi.status()` を確認し、接続完了かタイムアウトかを判定する。これにより、WiFi 接続中も他のモジュール（LED、センサー等）は正常に動作し続ける。

✓ リクエストフラグパターン

`requestConnect` はロジックフェーズからの「接続して」という指示。BuzzerModule の `requestPlay` と同じパターンで、`updateOutput()` が処理後に `false` に戻す。

13.3 ロジックフェーズでの使用例

```
void applyPattern(SystemData& data) {
    // ボタンが押されたらWiFi接続開始
    if (data.button.justPressed
        && !data.wifi.isConnected) {
        data.wifi.requestConnect = true;
    }

    // WiFi接続状態に応じた表示
    switch (data.wifi.state) {
        case WifiState::CONNECTING:
            snprintf(data.display.line1, 48, "WiFi: Connecting...");
            break;
        case WifiState::CONNECTED:
            snprintf(data.display.line1, 48,
                     "WiFi: OK (RSSI=%d)", data.wifi.rssi);
            break;
        case WifiState::FAILED:
            snprintf(data.display.line1, 48,
                     "WiFi: Failed (%d retries)",
                     data.wifi.retryCount);
            break;
        default: break;
    }
}
```

Listing 46: ロジックフェーズでの WiFi 制御

13.4 API 仕様

WifiModule — WiFi 接続管理モジュール

定義ファイル: `lib/WifiModule/WifiModule.h / .cpp`
分類: 入出力モジュール（`updateInput()` + `updateOutput()`）
対象デバイス: ESP32 WiFi
WiFi 接続のライフサイクル（接続・切断・再接続・タイムアウト）をステートマシンで自動管理するモジュール。ロジックフェーズからリクエストフラグで接続・切断を制御する。

13.4.1 機能概要

- 5 状態のステートマシンによる接続ライフサイクル管理
- 自動再接続（リトライ回数制限付き）
- 接続タイムアウト検出
- RSSI（信号強度）とローカル IP の取得
- リクエストフラグによる接続・切断制御
- `deinit()` による WiFi 切断

13.4.2 列挙型

◆ `WifiState`

WiFi 接続のステートマシン状態を定義する。

値	名前	説明
0	`DISCONNECTED`	未接続。`requestConnect` で接続開始
1	`CONNECTING`	接続試行中。タイムアウトまで待機
2	`CONNECTED`	接続済み。RSSI・IP を定期更新
3	`RECONNECTING`	再接続試行中。切断検出後に自動遷移
4	`FAILED`	接続失敗。リトライ上限超過またはタイムアウト

13.4.3 Config 構造体

◆ `WifiConfig`

WiFi 接続のパラメータを定義する。

メンバ	型	制約・既定値	説明
`ssid`	`const char*`	—	WiFi SSID
`password`	`const char*`	—	WiFi パスワード
`connectTimeoutMs`	`unsigned long`	例: 10000	接続タイムアウト [ms]
`reconnectIntervalMs`	`unsigned long`	例: 5000	再接続までの待機時間 [ms]
`maxRetries`	`uint8_t`	例: 5（0=無制限）	最大リトライ回数

13.4.4 Data 構造体

◆ `WifiData`

WiFi 接続の状態と制御フラグを保持する。

メンバ	型	初期値	説明
— 状態（入力フェーズで更新） —			
`state`	`WifiState`	`DISCONNECTED`	現在のステートマシン状態
`isConnected`	`bool`	`false`	接続中フラグ（簡易参照用）
`retryCount`	`uint8_t`	`0`	現在のリトライ回数
`rssi`	`int8_t`	`0`	信号強度 [dBm]（接続中のみ有効）
`localIp`	`uint32_t`	`0`	ローカル IP アドレス（uint32_t 形式）
— 制御（ロジックフェーズで設定） —			
`requestConnect`	`bool`	`false`	接続リクエスト。`true` で接続開始
`requestDisconnect`	`bool`	`false`	切断リクエスト。`true` で切断

13.4.5 メソッド

◆ `WifiModule(const WifiConfig& config)`

項目	内容
引数	`config` — `WifiConfig` への const 参照
動作	Config を `_config` にコピー保持する

◆ `bool init()`

項目	内容
戻り値	`bool` — 常に `true`
動作	WiFi モードを STA に設定し、自動再接続を無効化する（ステートマシンで管理するため）。状態を `DISCONNECTED` に初期化

◆ `void updateInput(SystemData& data)`

項目	内容
書き込み先	`data.wifi`
タイミング制御	毎ループ実行
動作	WiFi 接続状態を確認し、ステートマシンの状態遷移を処理。`state`・`isConnected`・`rssi`・`localIp`・`retryCount` を更新

◆ `void updateOutput(SystemData& data)`

項目	内容
読み取り元	`data.wifi`
タイミング制御	毎ループ実行
動作	`requestConnect` が `true` なら接続開始、`requestDisconnect` が `true` なら切断。各フラグを処理後に `false` にクリア

◆ `void deinit()`

項目	内容
動作	WiFi 接続を切断し、WiFi モードを OFF にする

▷ ステートマシン遷移

状態遷移の概要:

- DISCONNECTED → CONNECTING（requestConnect）
- CONNECTING → CONNECTED（接続成功）
- CONNECTING → FAILED（タイムアウト）
- CONNECTED → RECONNECTING（切断検出）
- CONNECTED → DISCONNECTED（requestDisconnect）
- RECONNECTING → CONNECTED（接続成功）
- RECONNECTING → FAILED（リトライ超過）
- FAILED → CONNECTING（requestConnect、リトライカウントリセット）

13.4.6 使用例

```
// ProjectConfig.h
const WifiConfig WIFI_CONFIG = {
    .ssid                 = "YourSSID",
    .password             = "YourPassword",
    .connectTimeoutMs     = 10000,  // 10秒タイムアウト
    .reconnectIntervalMs  = 5000,   // 5秒後に再接続
    .maxRetries           = 5,      // 最大5回リトライ
};

// ロジックフェーズでの使用
```

```
void applyPattern(SystemData& data) {
    // ボタンでWiFi接続トグル
    if (data.button.justPressed) {
        if (data.wifi.isConnected) {
            data.wifi.requestDisconnect = true;
        } else {
            data.wifi.requestConnect = true;
        }
    }
    // 接続中はRSSIを表示
    if (data.wifi.isConnected) {
        snprintf(data.display.line3, 48,
                 "RSSI: %d dBm", data.wifi.rssi);
    }
}
```

Listing 47: WifiModule の使用例

14 【応用】EncoderModule — 割り込み連携パターン

EncoderModule で学べるパターン

- ハードウェア割り込み（ISR）と volatile 変数の連携
- IRAM_ATTR による割り込みハンドラの RAM 配置
- ISR → volatile 変数→ updateInput() → SystemData のデータフロー
- deinit() による割り込みの解除

14.1 割り込みハンドラの設計

```
class EncoderModule : public IModule {
private:
    EncoderConfig _config;
    ModuleTimer   _sampleTimer;

    // ISRが更新する変数（volatile必須）
    volatile int32_t _isrCount = 0;

    // updateInput()内で使う前回値
    int32_t _prevCount = 0;

    // ISRはstatic関数でなければattachInterruptに渡せない
    static void IRAM_ATTR _isrHandlerA();
    static EncoderModule* _instance;

    void IRAM_ATTR _onPulseA();
    // ...
};
```

Listing 48: EncoderModule.h — 割り込み関連

▷ なぜ static が必要なのか？ — 初学者向け解説

C++ のクラスのメンバ関数（例: `_onPulseA()`）は、内部的に「どのオブジェクトの関数か」という情報（this ポインタ）を持っています。しかし、`attachInterrupt()` が受け付けるのは C 言語形式の「普通の関数ポインタ」だけで、this ポインタを渡す仕組みがありません。
そこで以下の 2 段構えで解決します。

1. **static 関数**を用意する — static 関数は this ポインタを必要としないので、割り込みに登録できる
2. **static なインスタンスポインタ**を用意する — static 関数の中から「どのオブジェクトか」を知るための橋渡し

```
// 1. staticポインタ: 「どのエンコーダか」を覚えておく
static EncoderModule* _instance;

// 2. static関数: 割り込みに登録できる
static void _isrHandlerA() {
    _instance->_onPulseA();  // ここで実際の処理に橋渡し
}

// init()で登録
_instance = this;
attachInterrupt(pin, _isrHandlerA, RISING);
```

これは「C++ のオブジェクト指向と C 言語の割り込みの橋渡し」パターンです。割り込みを使わない通常のモジュールでは不要なので、初めて見る場合はこのパターンだけ覚えれば十分です。

！ ISR（割り込みサービスルーチン）の制約

- `attachInterrupt()` に渡す関数は **static 関数**でなければならない
- ISR 内では `Serial.print()` やメモリ確保等の重い処理は禁止
- ESP32 等では `IRAM_ATTR` を付けて RAM に配置する必要がある（ボード依存）
- ISR から参照する変数は `volatile` を付ける

14.2 ISR → volatile → updateInput() の流れ

```
// ISRハンドラ（IRAM_ATTR = RAM配置、高速アクセス）
void IRAM_ATTR EncoderModule::_isrHandlerA() {
    if (_instance) {
        _instance->_onPulseA();
    }
}

void IRAM_ATTR EncoderModule::_onPulseA() {
    // A相の立ち上がり時にB相を読み取って方向を判定
    if (digitalRead(_config.pinB) == HIGH) {
        _isrCount++;    // CW
    } else {
        _isrCount--;    // CCW
    }
}

```

```
// updateInput()は入力フェーズで呼ばれる
void EncoderModule::updateInput(SystemData& data) {
    // volatile変数を一度だけ読み取り
    int32_t currentCount = _isrCount;

    data.encoder.count = currentCount;

    // 速度計算（一定間隔で実行）
    if (_sampleTimer.getNowTime() >= _config.sampleIntervalMs) {
        unsigned long elapsed = _sampleTimer.getNowTime();
        _sampleTimer.setTime();

        int32_t delta = currentCount - _prevCount;
        _prevCount = currentCount;

        float revolutions = (float)delta / _config.pulsesPerRev;
        float seconds = (float)elapsed / 1000.0f;
        data.encoder.rpm = (seconds > 0)
            ? (revolutions / seconds * 60.0f) : 0.0f;
        data.encoder.isValid = true;
    }
}
```

Listing 49: EncoderModule.cpp — ISR と update()

✓ volatile 変数を一度だけ読み取る

`int32_t currentCount = _isrCount;` で値をローカル変数にコピーしてから使う。volatile 変数を関数内で複数回参照すると、参照のたびに値が変わる可能性がある。一度だけ読み取ることで、同一 updateInput() 内での値の一貫性を保証する。

✓ BleModule との共通パターン

BleModule（コールバック）と EncoderModule（ISR）は、「メインタスク外からのデータ受け渡し」という同じ問題を解決している。どちらも **volatile 変数（またはフラグ）**を介して安全にデータを渡す。BLE はバッファ + フラグ、エンコーダはカウンタ変数という違いはあるが、原理は同じ。

14.3 deinit() で割り込みを解除

```
void EncoderModule::deinit() {
    detachInterrupt(digitalPinToInterrupt(_config.pinA));
    _instance = nullptr;
    Serial.println("[Encoder] deinit OK");
}
```

Listing 50: EncoderModule.cpp — deinit()

✓ 割り込み解除の重要性

割り込みを解除しないままスリープに入ると、スリープ中にも割り込みが発生してウェイクアップの原因になる可能性がある。また、`_instance = nullptr` にすることで、deinit 後に ISR が呼ばれても null チェックで安全にスキップされる。

▷ 複数インスタンスへの拡張

現在の実装は static ポインタ 1 つ（`_instance`）のため、エンコーダモジュールは 1 つしか使えない。複数使う場合は `static EncoderModule* _instances[MAX]` の配列に拡張し、チャネル ID でインデックスする方式に変更する。

14.4 API 仕様

EncoderModule — ロータリーエンコーダ入力モジュール

定義ファイル: `lib/EncoderModule/EncoderModule.h / .cpp`
分類: 入力モジュール（`updateInput()` のみオーバーライド）
対象デバイス: インクリメンタル型ロータリーエンコーダ（2 相出力）
ハードウェア割り込みで A 相パルスをカウントし、B 相の状態から回転方向を判定する。周期的に RPM（回転速度）を計算して `SystemData` に反映する。

14.4.1 機能概要

- ハードウェア割り込み（A 相）によるパルスカウント
- B 相による CW/CCW 方向判定
- 周期的 RPM 計算（`ModuleTimer` ベース）
- 累積パルス数の保持（正:CW、負:CCW）
- `deinit()` による割り込み解除

14.4.2 Config 構造体

◆ `EncoderConfig`

エンコーダの接続ピンと計測パラメータを定義する。

メンバ	型	制約・既定値	説明
`pinA`	`uint8_t`	—	エンコーダ A 相ピン（割り込み対応ピンを使用すること）
`pinB`	`uint8_t`	—	エンコーダ B 相ピン
`pulsesPerRev`	`uint16_t`	例: 20	1 回転あたりのパルス数
`sampleIntervalMs`	`unsigned long`	例: 50	RPM 計算間隔 [ms]

14.4.3 Data 構造体

◆ EncoderData

エンコーダの計測結果を保持する。

メンバ	型	初期値	説明
count	int32_t	0	累積パルス数。正:CW（時計回り）、負:CCW（反時計回り）
rpm	float	0.0f	回転速度 [rpm]
isValid	bool	false	RPM 計算が 1 回以上行われたら true

14.4.4 メソッド

◆ EncoderModule(const EncoderConfig& config)

項目	内容
引数	config — EncoderConfig への const 参照
動作	Config を_config にコピー保持する

◆ bool init()

項目	内容
戻り値	bool — 常に true
動作	A 相・B 相ピンを INPUT_PULLUP で設定。A 相に RISING エッジの割り込みハンドラを登録。シングルインスタンス参照（_instance）を自身に設定

◆ void updateInput(SystemData& data)

項目	内容
書き込み先	data.encoder
タイミング制御	ModuleTimer で_config.sampleIntervalMs ごとに RPM を計算
動作	ISR が更新した volatile カウンタを読み取り、前回値との差分から RPM を計算。count・rpm・isValid を更新

◆ void deinit()

項目	内容
動作	A 相ピンの割り込みを detachInterrupt() で解除する

▷ RPM 計算式

RPM 計算は以下の式で行う:

$$RPM = \frac{\Delta count}{pulsesPerRev} \times \frac{60000}{sampleIntervalMs}$$

ここで $\Delta count$ はサンプリング間隔内のパルス数変化量である。

！シングルインスタンス制約

`attachInterrupt()` に static 関数を渡すため、内部で `static EncoderModule* _instance` を使用している。同時に使用できる EncoderModule のインスタンスは **1 つのみ**である。複数エンコーダが必要な場合は ISR ディスパッチの仕組みを拡張する必要がある。

14.4.5 使用例

```
// ProjectConfig.h
const EncoderConfig ENCODER_CONFIG = {
    .pinA             = 39,  // GPIO39: A相
    .pinB             = 40,  // GPIO40: B相
    .pulsesPerRev     = 20,  // 1回転20パルス
    .sampleIntervalMs = 50,  // 50msごとに速度計算
};

// ロジックフェーズでの使用
void applyPattern(SystemData& data) {
    if (data.encoder.isValid) {
        float speed = data.encoder.rpm;
        // 回転速度に応じた制御
    }
}
```

Listing 51: EncoderModule の使用例